ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ - VIỄN THÔNG

ĐỒ ÁN CHUYÊN NGÀNH

ĐỀ TÀI:
HỆ THỐNG GIÁM SÁT, ĐỊNH VỊ ÁP DỤNG CHO HOẠT ĐỘNG CỨU HỘ, CỨU NẠN SỬ DỤNG CÔNG NGHỆ TRUYỀN THÔNG LORAWAN



SINH VIÊN THỰC HIỆN:
NGUYỄN VĂN SƠN – 18DT1
ĐẶNG PHƯỚC NHẬT – 18DT1

GIÁO VIÊN HƯỚNG DẪN:
Ths.VŨ VÂN THANH

Đà Nẵng, 2022

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ - VIỄN THÔNG

CỘNG HÒA XÃ HÔI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

NHIỆM VỤ ĐỒ ÁN CHUYÊN NGÀNH

Họ tên sinh viên: Nguyễn Văn Sơn — Mã số sinh viên: 106180047
Lớp: 18DT1 Khoa: Điện Tử - Viễn Thông — Ngành: Kỹ Thuật Điện Tử
Họ tên sinh viên: Đặng Phước Nhật — Mã số sinh viên: 106170074
Lớp: 17DT2 Khoa: Điện Tử - Viễn Thông — Ngành: Kỹ Thuật Điện Tử

1. *Tên đề tài đồ án:*
   Hệ giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn sử dụng công nghệ truyền thông LoRaWan.
2. *Họ tên người hướng dẫn:* Ths. Vũ Vân Thanh

Đà Nẵng, ngày 24 tháng 6 năm 2022

Trưởng Bộ môn……

Người hướng dẫn

Ths. Vũ Vân Thanh

LỜI NÓI ĐẦU

LoRaWAN (Long Range Wireless Area Network) là một mạng diện rộng công suất thấp (LPWAN) dành cho những vật hoạt động bằng pin không dây trong mạng khu vực, quốc gia hoặc toàn cầu. Mạng lưới LoRaWAN hiện có mặt ở hơn 100 quốc gia và rất nhiều nhà khai thác lớn trên toàn thế giới giúp tạo ra một mạng lưới LoRaWAN phổ biến với những nước trên thế giới đang phát triển các thành phố thông minh, đô thị thông minh. Việt Nam cũng đã cho phép ứng dụng LoRa trong lĩnh vực IoT với tần số hợp pháp là 923MHz. Đây sẽ là một bước đi quan trọng trong việc ứng dụng IoT trong đời sống và phát triển của chúng ta sau này.

Nhận thấy tiềm năng phát triển khi ứng dụng mạng lưới LoRaWAN vào đời sống hiện nay. Là sinh viên khoa Điện Tử Viễn Thông của trường Đại học Bách Khoa- Đà Nẵng, với những kiến thức tôi được cung cấp từ thầy cô đã được tích luỹ và rèn luyện cùng với đó là việc mong muốn được đem kiến thức của mình để áp dụng thêm nhiều vào nhu cầu đời sống. Tôi đã thiết kế một ý tưởng về ứng dụng công nghệ truyền thông không dây lorawan vào hệ thống giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn.

Tôi xin cảm ơn đến các thầy cô trong khoa, đặc biệt là lời cảm ơn sâu sắc đến thầy Vũ Vân Thanh đã tận tình định hướng, hướng dẫn và cung cấp kiến thức để giúp tôi hoàn thành đồ án này.

LỜI CAM ĐOAN

Đồ án này là do chúng em tự thực hiện dựa vào một số tài liệu trước đó và không sao chép từ tài liệu hay công trình đã có trước đó. Các kết quả công bố trong Đồ Án Chuyên Ngành "Hệ giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn sử dụng công nghệ truyền thông LoRaWan" là trung thực và không sao chép từ công trình nào khác.

LỜI CẢM ƠN

“Báo cáo Đồ Án Chuyên Ngành” được viết dưới sự hướng dẫn của thầy Vũ Vân Thanh, Khoa Điện tử Viễn thông, Đại học Bách Khoa Đà Nẵng.

Nhân cơ hội này, chúng em xin gửi lời cảm ơn sâu sắc tới người hướng dẫn của chúng em, thầy Vũ Vân Thanh, đã tận tình hướng dẫn và động viên không ngừng trong suốt thời gian chúng em thực hiện Đồ Án Chuyên Ngành này.

Cuối cùng, chúng em xin gửi lời cảm ơn đến toàn thể thầy cô khoa Điện tử Viễn thông đã nhiệt tình giúp đỡ chúng em trong thời gian qua.

Đà Nẵng, ngày 24 tháng 06 năm 2022

Nguyễn Văn Sơn

Đặng Phước Nhật

MỤC LỤC

LỜI NÓI ĐẦU 3
LỜI CAM ĐOAN 4
LỜI CẢM ƠN 5
`CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 9
1.1 Giới thiệu chương 9
1.2 Tổng quan về Mạng truyền thông không dây LoRaWan 9
1.3 Tổng quan về hệ thống 10
1.4 Tính cấp thiết của đề tài 10
1.5 Giải pháp đề xuất 10
1.6 Giới hạn đề tài 10
1.7 Kết luận chương 11
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 12
2.1 Giới thiệu chương 12
2.2 Tổng quan về hệ thống 12
2.3 LoRaWan 13
2.3.1 Kiến trúc LoRaWan 13
2.3.2 Các lớp thiết bị 14
2.3.3 Kiểu điều chế 15
2.4 Chọn lựa linh kiện 18
2.4.1 Khối vi điều khiển lý 18
2.4.2 Khối nguồn 18
2.4.3 Khối định vị 19
2.5 Tìm hiểu các thành phần trong hệ thống 20
2.5.1 Vi điều khiển 20
2.5.2 Module LoRa 21
2.5.3 Định vị GPS Neo 7M 22
2.6 Giao tiếp UART 23

2.7 Giao tiếp SPI 26
2.8 Datacake 28
2.9 Kết thúc chương 29
CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ HỆ THỐNG VÀ KẾT QUẢ, ĐÁNH GIÁ 30
3.1 Giới thiệu chương 30
3.2 Yêu cầu đặt ra cho hệ thống 30
3.3 Tính toán thiết kế phần cứng end node 30
3.3.1 Yêu cầu thiết kế và sơ đồ khối 30
3.3.2 Tính toán thiết kế 30
3.4 Layout mô hình mạch và sản phẩm thực tế 35
3.4.2 Sản phẩm mạch thực tế 36
3.5 Gateway 37
3.6 Thiết kế phần mềm 38
3.6.1 Thiết kế chương trình node cảm biến 38
3.6.2 Hệ thống phần mềm 38
3.7 Kết quả Demo thực nghiệm 43
3.8 Đánh giá kết quả: 44
3.9 Kết thúc chương. 45
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 46
THAM KHẢO 47
PHỤ LỤC 1 48

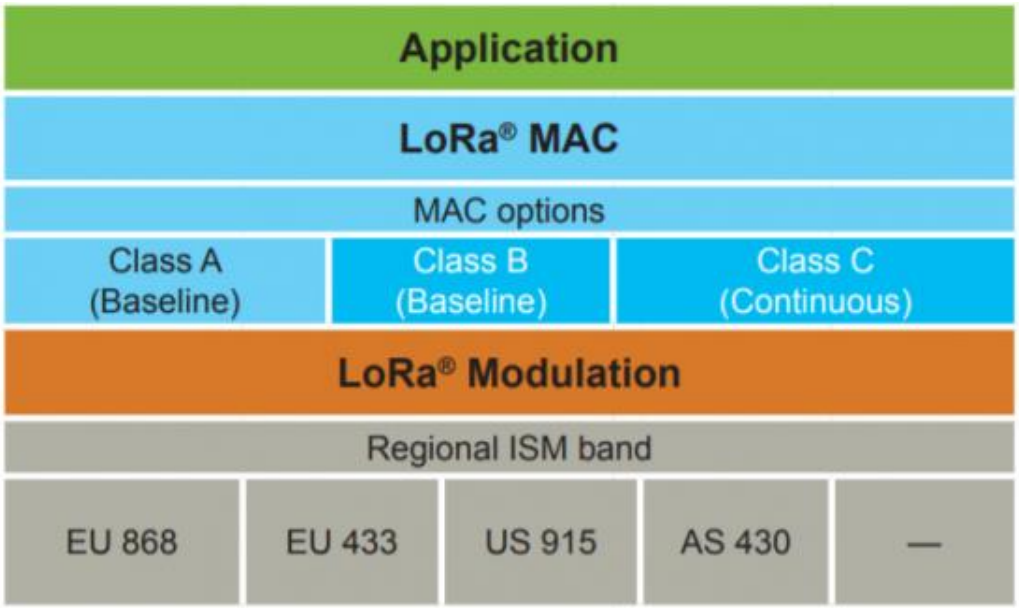
`CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu chương

Chương 1 sẽ trình bày làm rõ những phần sau: Tổng quan về mạng truyền thông không dây LoRaWan trong đó sẽ trình bày về khái niệm Mạng LoRaWan và những ứng dụng của nó trong đời sống của con người. Sau đó chương sẽ giới thiệu tổng quan về hệ thống giám sát, định vị áp dụng trong hoạt động cứu hộ, cứu nạn, từ đó sẽ đặt ra vấn đề cho đề tài này, đưa ra các mục đích và phương pháp nghiên cứu, những giới hạn của đề tài.

1.2 Tổng quan về Mạng truyền thông không dây LoRaWan

LoRaWAN được định nghĩa là một kiến trúc hệ thống và giao thức truyền thông trong mạng, trong khi đó LoRa là lớp vật lý (physical layer) cho phép thiết lập các kết nối truyền thông tầm xa. Giao thức và kiến trúc mạng có ảnh hưởng lớn nhất đến việc xác định thời lượng pin của một nút, dung lượng mạng, chất lượng dịch vụ, bảo mật và nhiều loại ứng dụng được phục vụ bởi mạng.



Hinh 1. 1: LoRa Protocol Stack

Ứng dụng của Mạng LoRaWan:

- Các hệ thống thành phố thông minh, với các cảm biến đậu đỗ xe, quản lý rác thải và sâu hơn nữa là những ngôi nhà thông minh. Những ngôi nhà này trang bị những cảm

biến phát hiện cháy nổ, khói cũng như các thông số môi trường như nhiệt độ, độ ẩm, khí gas trong nhà và tự đưa ra được những quyết định bảo vệ ngôi nhà.

- Các hệ thống nông nghiệp thông minh.
- Các hệ thống quan trắc môi trường, cảnh báo.
- Các hệ thống theo dõi, định vị.
- Các hệ thống đo lường từ xa như công tơ nước, điện.

1.3 Tổng quan về hệ thống

Trong đồ án này, hệ thống này sẽ quan sát được vị trí của người bị nạn. Nó sử dụng một bộ định vị GPS để truyền tín hiệu, sau đó gửi dữ liệu đến máy chủ qua mạng LoRaWan. Bằng cách sử dụng hệ thống Truyền dẫn không dây LoRaWan, các dữ liệu có thể truyền đi với một khoảng cách xa lên tới 15Km với một năng lượng thấp giúp hệ thống hoạt động lâu hơn mà không cần nạp lại năng lượng cho thiết bị. Do đó, sự ổn định và hiệu suất của truyền nhận dữ liệu thuận lợi hơn ở khoảng cách xa.Nhìn chung, hệ thống này sẽ nâng cao hiệu quả giám sát và độ chính xác.

Mục đích chính của đồ án này là giúp cho quá trình quan sát được vị trí của người gặp nạn, giúp cho việc cứu hộ trỡ nên dễ dàng hơn.

1.4 Tính cấp thiết của đề tài

Với ngành du lịch, cũng như việc khám phá tự nhiên phát triển mạnh như hiện nay, thì sự cố của con người thường xuyên xảy ra.

Đội công tác cứu hộ, cứu nạn mất nhiều thời gian khi không xác định được vị trí, phạm vị cụ thể của người bị nạn.

1.5 Giải pháp đề xuất

Sử dụng bộ định vị GPS và cùng với việc sử dụng công nghệ truyền thông không dây LoRaWan để truyền nhận dữ liệu ở khoảng các xa.Thông qua Web App Datacake để biết được vị trí, phạm vi người bị nạn.

Hệ thống giám sát từ xa là giải pháp hiệu quả cho đội cứu hộ, cứu nạn không mất quá nhiều thời gian để tìm kiếm người bị nạn. Từ đó đạt được độ chính xác và kịp thời cứu nạn.

1.6 Giới hạn đề tài

Với những gì đã trình bày như trên, chúng tôi đã tiến hành nghiên cứu, khảo sát và thực hiện và dự kiến đạt được những mục tiêu như sau:

- Thiết kế và thi công mạch.
- Thiết hế và thi công App sever - Datacake.
- Xác định tọa độ địa lí của bộ định vị GPS khi phát tín hiệu
- Gửi dữ liệu đo đến máy chủ bằng mạng LoRaWan hàng ngày trong vòng 6 phút đến 10 phút.

1.7 Kết luận chương

Chương 1 đã trình bày ra được tổng quan về hệ thống giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn bằng mạng LoRaWan. Sau khi đã nêu lên được những khái niệm, nguyên tắc hoạt động, chương đã đặt ra được vấn đề cần thiết, từ đó cũng đưa ra được những mục đích, phương pháp nghiên cứu và giới hạn của đề tài.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Giới thiệu chương

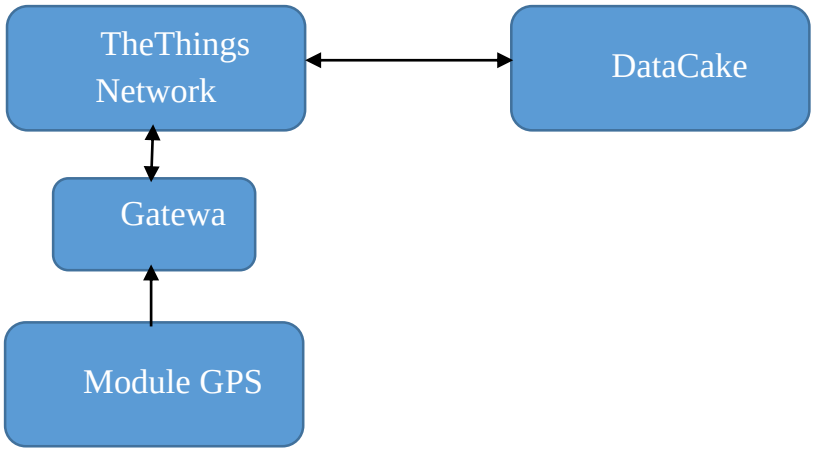
Chương 2 sẽ trình bày những cơ sở lý thuyết liên quan đến hệ thống giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn bằng công nghệ mạng LoRaWan. Sau đó sẽ đưa ra một số linh kiện thiết bị, những phần mềm nền tảng có thể được sử dụng để thiết kế xây dựng nên đề tài này. Những cơ sở lý thuyết này sẽ là nền tảng để phân tích tính toán thiết kế xây dựng ở Chương 3.

2.2 Tổng quan về hệ thống

Sau khi điều tra tổng quan tính cấp thiết của đề tài nhóm đã tìm hiểu các tài liệu [1], [2], [3], [4] để quyết định thực hiện một hệ thống giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn bằng công nghệ truyền thông không dây LoRaWan. Hệ thống sẽ theo dõi vị trí của người bị nạn. Hệ thống này là sự kết hợp của một LoRaServer, một LoRaGateWay và bộ định vị GPS. Trước hết, bộ định vị sẽ phát tín hiệu, sau đó gửi về server thông qua LoRaGateWay bằng giao thức LoRaWan, từ LoRaServer đưa dữ liệu sang Web App (DataCake) để hiện thị vị trí trên bản đồ.

Hệ thống giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn có 4 bộ phận chính:

- Bộ định vị GPS.
- LoRaGateWay.
- LoRaServer.
- Web App DataCake.

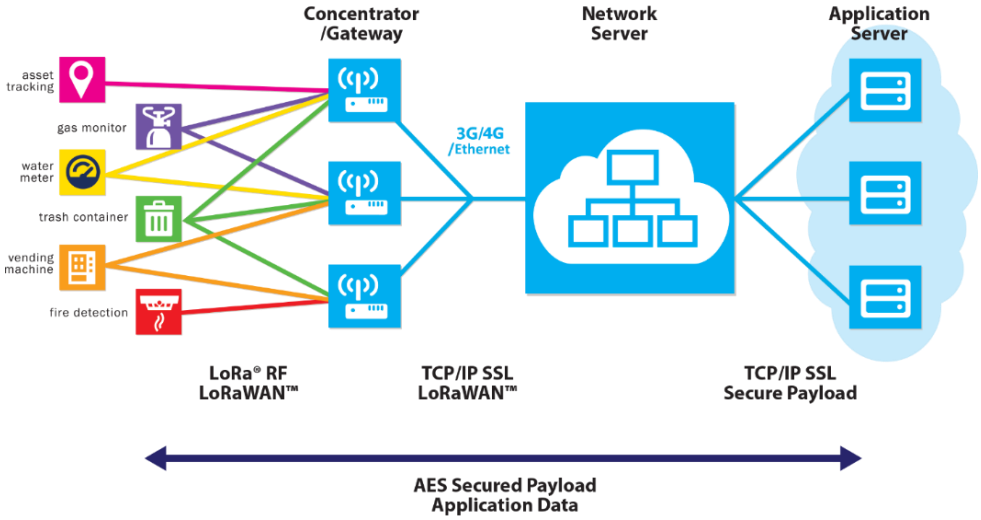


Hình 2. 1: Sơ đồ khối của hệ thống

2.3 LoRaWan

2.3.1 Kiến trúc LoRaWan

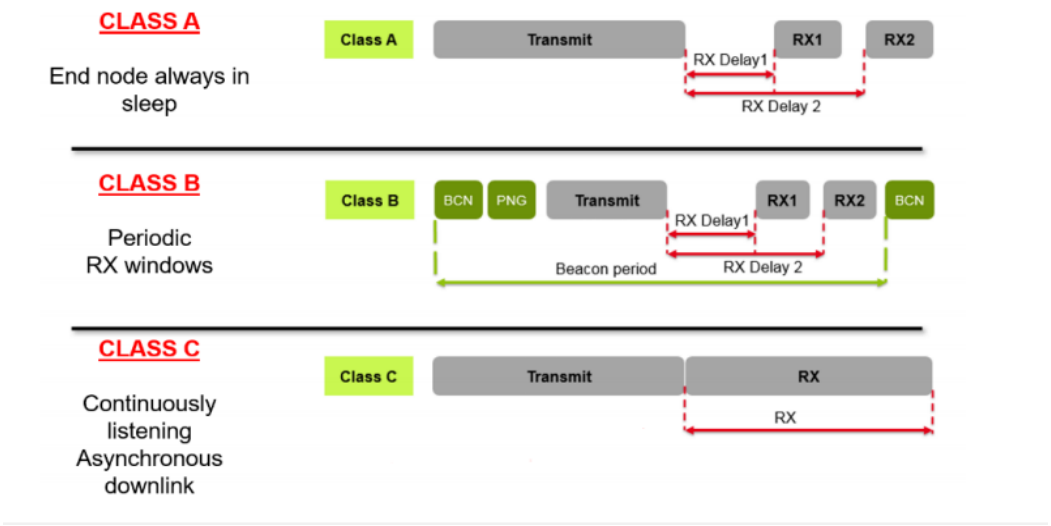
Hình dưới đây cho chúng ta một ví dụ điển hình về việc triển khai mạng LoRaWan từ đầu tới cuối:



Hình 2. 2: Kiến trúc hệ thống LoRaWan

Các thành phần của mạng LoRaWAN bao gồm:

- End Devices (thiết bị cuối) hỗ trợ LoRaWAN: là một cảm biến hoặc thiết bị truyền động được kết nối không dây với mạng LoRaWAN thông qua các gateway sử dụng công nghệ điều chế LoRa.
- Gateway LoRaWAN (cổng LoRaWAN): nhận các dữ liệu RF được điều chế LoRa từ các thiết bị cuối và chuyển tiếp dữ liệu này đến máy chủ ở mạng LoRaWAN. Các cảm biến được kết nối với gateway thông qua mạng IP backbone.
- Network server (máy chủ mạng): quản lý toàn bộ hệ thống mạng, các thông số thích hợp để điều chỉnh hệ thống và thiết lập kết nối AES 128-bit an toàn để truyền tải và kiểm soát dữ liệu. Máy chủ mạng đảm bảo tính xác thực của mọi cảm biến trên mạng và tính toàn vẹn của các thông báo, tuy nhiên lại không thể nhìn thấy hoặc truy cập vào dữ liệu ứng dụng.
- Application servers (máy chủ ứng dụng): chịu trách nhiệm xử lý, quản lý và diễn giải dữ liệu nhận được từ các cảm biến một cách an toàn, đồng thời tạo ra một downlink payloads tới các thiết bị đầu cuối.



Hình 2. 3: Các chế độ hoạt động cả thiết bị

2.3.2 Các lớp thiết bị

LoRaWAN có ba loại thiết bị điểm cuối khác nhau để đáp ứng các nhu cầu khác nhau:

Class A - Thiết bị đầu cuối hai chiều, công suất thấp nhất:

Giao tiếp lớp A luôn được thiết bị đầu cuối khởi tạo và hoàn toàn không đồng bộ. Mỗi lần truyền đường lên có thể được gửi bất cứ lúc nào và được theo sau bởi hai cửa sổ đường xuống ngắn, tạo cơ hội cho giao tiếp hai chiều hoặc các lệnh điều khiển mạng nếu cần. Đây là một loại giao thức ALOHA. Thiết bị cuối có thể vào chế độ ngủ công suất thấp. Điều này làm cho lớp A trở thành chế độ hoạt động công suất thấp nhất, trong khi vẫn cho phép giao tiếp đường lên bất cứ lúc nào.

Class B - Thiết bị đầu cuối hai hướng với độ trễ đường xuống xác định:

Ngoài các cửa sổ nhận được khởi tạo như lớp A, các thiết bị lớp B được đồng bộ hóa với mạng bằng cách sử dụng các báo hiệu định kỳ và mở các 'khe ping' đường xuống vào các thời điểm đã lên lịch. Điều này cung cấp cho mạng khả năng gửi thông tin liên lạc đường xuống với độ trễ xác định, nhưng sẻ tiêu thụ năng lượng lớn hơn lớp A. Độ trễ có thể lập trình lên đến 128 giây để phù hợp với các ứng dụng khác nhau và mức tiêu thụ năng lượng thấp để vẫn có hiệu lực cho các ứng dụng sử dụng pin.

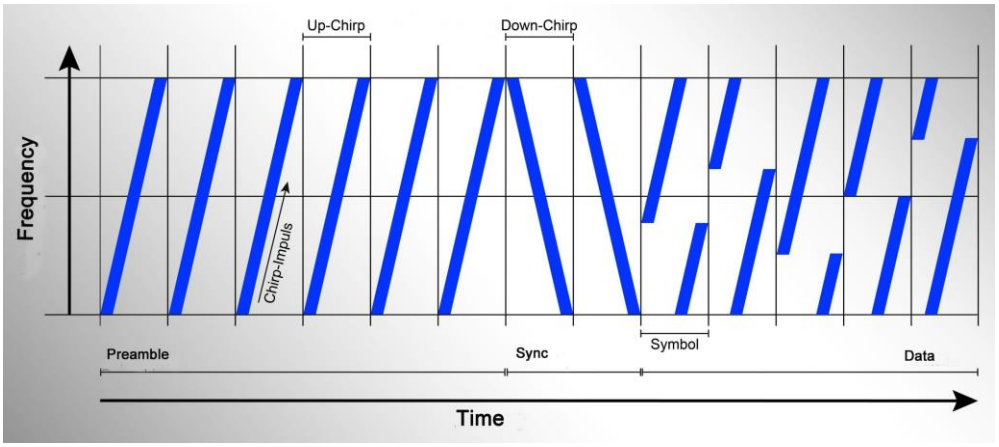
Class C - Thiết bị đầu cuối hai chiều, có độ trễ thấp nhất:

Ngoài cấu trúc lớp A của đường lên theo sau là hai cửa sổ đường xuống, lớp C còn làm giảm độ trễ trên đường xuống bằng cách giữ cho đầu thu của thiết bị đầu cuối luôn mở mà thiết bị không truyền (bán song công). Dựa trên điều này, máy chủ mạng có thể bắt đầu truyền đường xuống bất kỳ lúc nào với giả định rằng bộ thu thiết bị đầu cuối đang mở, do đó không có độ trễ. Mức tiêu hao năng lượng của bộ thu cao (lên đến ~ 50mW) và vì vậy lớp C phù hợp cho các ứng dụng có nguồn điện liên tục. Đối với các

thiết bị chạy bằng pin, có thể chuyển đổi chế độ tạm thời giữa các lớp A & C và rất hữu ích cho các tác vụ không liên tục, chẳng hạn như cập nhật chương trình cơ sở qua mạng.

2.3.3 Kiểu điều chế

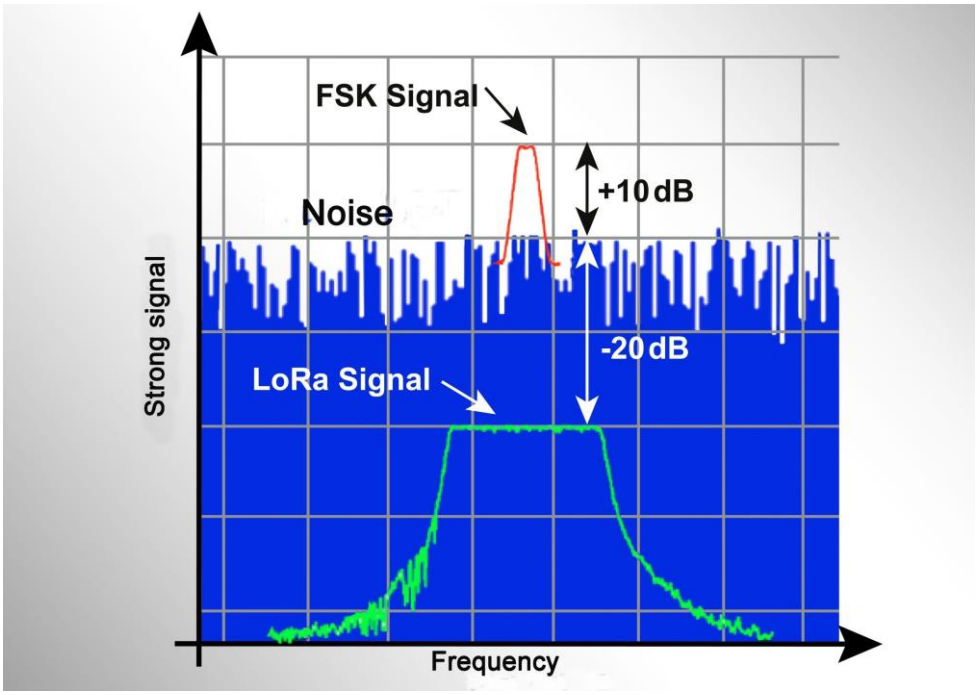
LoRa là một kỹ thuật điều chế có nguồn gốc từ kỹ thuật "Chirp Spread Spectrum" (CSS), trong đó Chirp là viết tắt của "Compressed High Intetsity Radar Pulse", là một kỹ thuật sử dụng rất phổ biến trong sonar và radar. Tín hiệu Chirp có biên độ không đổi và tần số thay đổi, nếu tần số thay đổi từ tần số thấp nhất đến tần số cao nhất thì được gọi là up-chirp và thay đổi từ tần số cao nhất đến thấp nhất được gọi là down-chirp.



Hình 2. 4: Tín hiệu Chirp

- **Tính chất đặc biệt:**

➢ Vì LoRa hoạt động trong các dải tần số ISM (433 MHz, 868 MHz và 915 MHz), công suất truyền bức xạ bị hạn chế. Để có phạm vi vô tuyến lớn hơn các loại điều chế thông thường như Để đạt được FSK (Tần số chuyển keying), độ nhạy của máy thu đã được cải thiện đáng kể với LoRa. Người nhận LoRa vẫn có thể nhận và giải mã thành công tín hiệu LoRa hữu ích lên đến 20 dB dưới mức tiếng ồn, dẫn đến độ nhạy của máy thu tối đa là -149 dBm. So với độ nhạy FSK tối đa khoảng. Cỗ máy 125 dBm đến -130 dBm, LoRa cung cấp một cải tiến đáng kể. Với máy thu FSK, tín hiệu chỉ có thể được giải mã thành công nếu tín hiệu hữu ích xấp xỉ.

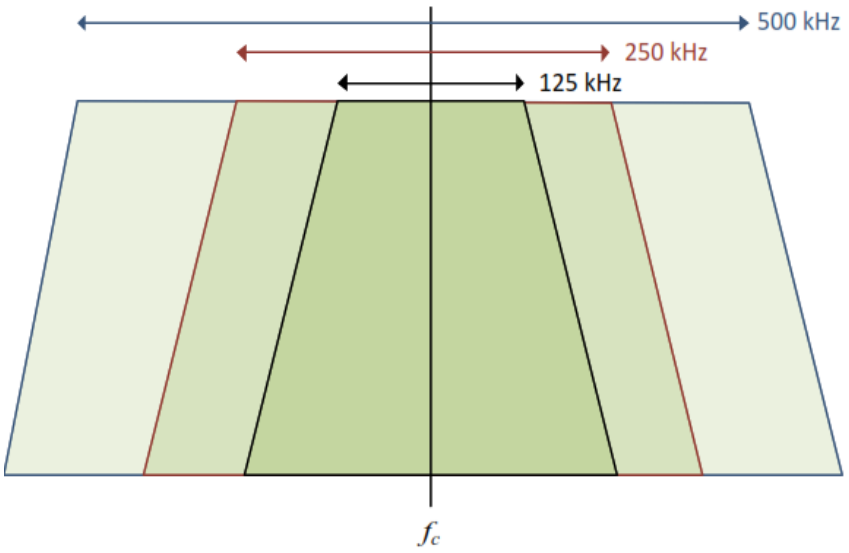


Hình 2. 5: So sánh mức tín hiệu LoRa với FSK

- Độ mạnh của nhiễu sóng radio tốt hơn đáng kể so với FSK. Hệ thống FSK chỉ hoạt động chính xác nếu tín hiệu nhiễu ít nhất là 10 dB yếu hơn tín hiệu hữu ích. Trong trường hợp tốt nhất, Các hệ thống LoRa vẫn có thể nhận được tín hiệu hữu ích nếu tín hiệu nhiễu là 20 dB mạnh hơn tín hiệu hữu ích.

- **Các thông số của LoRa:**

- Khi truyền đi một gói tin, thì hai thông số chúng ta quan tâm đó là khoảng cách truyền (range) và tốc độ truyền (Data rate). Đối với LoRa thì 2 giá trị trên phụ thuộc vào 3 thông số có thể điều chỉnh được: Băng thông (BW), hệ số trải phổ(SF) và tốc độ mã hóa(CR).
- **BW** xác định biên độ tần số mà tín hiệu chirp có thể thay đổi. Các chip LoRa khác nhau sẽ cho phép tùy biến cấu hình các mức băng thông khác nhau, nhưng thông thường sẽ cấu hình ba mức băng thông phổ biến là 125kHz, 250kHz và 500 kHz. Băng thông cao sẽ cho phép mã hóa tín hiệu nhanh hơn, giúp thời gian truyền dữ liệu nhanh hơn nhưng bù lại khoảng cách truyền cũng sẽ ngắn đi.



Hình 2. 6: Băng thông của LoRa

- **SF** xác định số lượng tín hiệu chirp khi mã hóa tín hiệu đã được điều chế tần số (chipped signal) của dữ liệu, SF là các giá trị nguyên từ 7 đến 12. Ví dụ nếu SF = 12 có nghĩa là 1 mức logic của tín hiệu sẽ được mã hóa bởi 12 xung tín hiệu chirp. Giá trị SF càng lớn thì thời gian truyền dữ liệu sẽ lâu hơn nhưng đổi lại tỉ lệ lỗi bit BER sẽ giảm và khoảng cách truyền cũng sẽ xa hơn.
- **CR** là số lượng bit được tự thêm vào trong payload gói tin LoRa để mạch nhận có thể sử dụng để phục hồi lại một số bit dữ liệu đã nhận sai và từ đó phục hồi được nguyên vẹn dữ liệu payload. CR là các giá trị nguyên từ 1 đến 4 và thường biểu thị ở dạng **4/CR+4** (ví dụ: 4/5, 4/6, 4/7, 4/8). Do đó, sử dụng CR càng cao thì khả năng nhận dữ liệu đúng càng tăng, nhưng bù lại chip LoRa sẽ phải gửi nhiều dữ liệu hơn và làm tăng thời gian truyền.
- Tóm lại tùy vào mục đích ứng dụng ưu tiên hơn về khoảng cách hay tốc độ truyền mà chúng ta có thể cấu hình các tham số BW, SF, CR phù hợp. Ví dụ nếu mục đích của ứng dụng là thu thập các giá trị của cảm biến cần truyền ở khoảng cách xa không quá yêu cầu về thời gian, vì vậy sẽ không yêu cầu băng thông lớn nên chúng ta sẽ cấu hình các hệ số BW thấp (125 kHz), SF cao (12) và CR có thể tùy chỉnh.

- Lora vẫn duy trì được các đặc tính công suất thấp giống như điều chế FSK ngoài ra còn tăng đáng kể phạm vi truyền thông. Chirp Spread Spectrum đã được sử dụng trong lĩnh vực quân sự và vũ trụ trong nhiều thập kỷ do khoảng cách giao tiếp tầm xa có thể đạt được và khả năng can thiệp mạnh mẽ, và LoRa cũng là ứng dụng với mức chi phí thấp đầu tiên được sử dụng cho mục đích thương mại.
- Long Range (LoRa) Lợi thế nhất của LoRa là ở khả năng tầm xa của công nghệ. Một gateway hoặc một trạm gốc có thể bao phủ toàn bộ thành phố hoặc hàng trăm km vuông. Phạm vi phụ thuộc nhiều vào môi trường hoặc vật cản tại một vị trí nhất định, nhưng LoRa và LoRaWAN có tầm liên kết (Link Budget) rộng hơn so với bất kỳ công nghệ truyền thông tiêu chuẩn nào khác. Tầm liên kết, thường được tính bằng

decibel (dB), là yếu tố chính để xác định phạm vi liên kết trong một môi trường nhất định.

2.4 Chọn lựa linh kiện

Như đã trình bày ở Chương 1, đề tài sẽ xây dựng 1 hệ thống giám sát, định vị áp dụng cho hoạt động cứu hộ, cứu nạn có các chức năng sau:

- Phát tín hiệu định vị từ người bị nạn
- Thu thập thông số, tọa độ.
- Thu thập khoảng cách vị trí người bị nạn tới trạm cứu hộ.
- Gửi dữ liệu đọc được lên Thethingsnetwork.
- Đưa dữ liệu đến Datacake.

Từ các chức năng đó, chúng ta xét tổng quan để lựa chọn các linh kiện phù hợp.

2.4.1 Khối vi điều khiển lý

Nhiệm vụ:

Nhận và xử lý dữ liệu thu được từ các cảm biến để hiển thị lên màn hình đồng thời gửi dữ liệu lên server hoặc nhận tín hiệu điều khiển từ server.

Chọn lựa linh kiện:

Để đáp ứng các yêu cầu như trên Chương 1 đề ra thì ta nên chọn các loại vi điều khiển có khả năng tiết kiệm năng lượng, đơn giản, arduino**.**

2.4.2 Khối nguồn

Nhiệm vụ:

Mạch tổng thể yêu cầu sử dụng 1 nguồn chính đó là nguồn 12V để cung cấp nguồn 3.3V cho module định vị và cấp nguồn vi điều khiển và module LoRa.

Nguồn 3.3V:

- Điện áp vào: 12VDC.
- Điện áp ra: 3.3VDC (3.1V-3.5V) sai số 0.1-0.2V.

Chọn lựa linh kiện:

Khối cấp nguồn:

- Để tạo nguồn 3.3Vthì có nhiều IC như LM2596ADJ, LM7812, L7905CV:
- LM2596 đây là IC ổn áp dạng xung tạo ra nguồn có thể điều chỉnh được với dòng tối đa lên đến 3A.
- LM7805 là ổn áp sử dụng phổ biến trong các mạch điều khiển có thể tạo ra nguồn một chiều 5V/1A.
- L7905 là IC thuộc họ IC ổn áp của họ 79XX. Chuyên dùng để ổn áp nguồn âm hay nguồn đối xứng. Đối với IC7905 thì điện áp được ổn định -3.3V so với đầu vào chân so sánh. Điện áp đầu vào lớn hơn điện áp đầu ra và chỉ ổn định trong một điện áp cố định là -3.3V.

- Vậy sau khi tìm hiểu trên thị trường, chúng tôi có một số nhận xét sau:
- Không chọn LM7805 vì dòng ra cực đại của IC7805 cực đại chỉ có 1A so với lý thuyết nhưng thực tế thì còn khoảng (200-300) mA.
- Không chọn LM7905 vì với yêu cầu của mạch thì ta cũng không cần dùng nguồn đối xứng.
- Chọn linh kiện LM2576-3.3V vì thông dụng trên thị trường, dễ thiết kế với chi phí thấp mà chất lượng nguồn đầu ra lại ổn định dòng ngõ ra lớn phù hợp với yêu cầu đặt ra.

2.4.3 Khối định vị

Nhiệm vụ:

Ở chương 1 về giới thiệu đề tài thì qua đó, hệ thống sẽ sử dụng các loại linh kiện sau:

- Module GPS – Neo – 7M
- An-ten

Chọn lựa linh kiện:

- Module GPS:

Module GPS U-blox NEO-7M-0-000 rất cần thiết cho các dự án định vị vị trí và chuyển động, tốc độ cập nhật rất nhanh, trả về tọa độ có độ chính xác rất cao, kết nối và sử dụng rất đơn giản là những ưu điểm của loại module này.



Hình 2. 7: Module GPS Neo 7M

- Anten GPS:

Anten GPS là loại anten nhận sóng cao kể cả những lúc điều kiện thời tiết không tốt



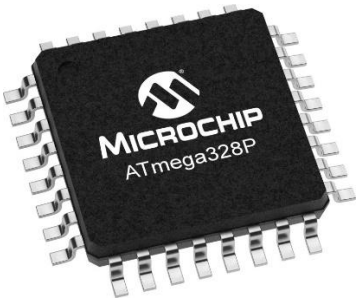
Hình 2. 8: Anten GPS

2.5 Tìm hiểu các thành phần trong hệ thống

2.5.1 Vi điều khiển

2.5.1.1 Atemegta328p

Giới thiệu:



Hình 2.9: Atemega328p

Bộ vi điều khiển dựa trên Microchip picoPower® 8-bit AVR® RISC hiệu suất cao kết hợp bộ nhớ 32 KB ISP Flash với khả năng đọc trong khi ghi, 1024B EEPROM, 2 KB SRAM, 23 dòng I / O mục đích chung, 32 thanh ghi làm việc mục đích chung , ba bộ đếm / bộ hẹn giờ linh hoạt với các chế độ so sánh, ngắt bên trong và bên ngoài, USART có thể lập trình nối tiếp, giao diện nối tiếp Hai dây hướng byte, cổng nối tiếp SPI, bộ chuyển đổi A / D 6 kênh 10 bit (8 kênh trong TQFP và gói QFN / MLF), bộ đếm thời gian cơ quan giám sát có thể lập trình với bộ dao động bên trong và năm chế độ tiết kiệm năng lượng có thể lựa chọn phần mềm. Thiết bị hoạt động trong khoảng 1,8-5,5 volt.

Bằng cách thực hiện các lệnh mạnh mẽ trong một chu kỳ xung nhịp duy nhất, thiết bị đạt được thông lượng đạt tới một MIPS trên mỗi MHz, cân bằng giữa mức tiêu thụ điện năng và tốc độ xử lý.

Phần mềm lập trình: Arduino IDE

2.5.2 Module LoRa

Giới thiệu:



Hình 2. 10: Module LoRa RF96

- Bộ thu phát RFM95W theo chuẩn LoRaTM, cung cấp giao tiếp phổ rộng cực xa và khả năng chống nhiễu cao trong khi giảm thiểu mức tiêu thụ năng lượng đến mức cực thấp.
- Sử dụng kỹ thuật điều chế LoRaTM được cấp bằng sáng chế của HopeRF, RFM95W có thể đạt được độ nhạy trên -148dBm chỉ với thạch anh và linh kiện có giá thành thấp. Độ nhạy cao kết hợp với tích hợp bộ khuếch đại công suất +20 dBm khiến nó phù hợp, tối ưu cho mọi ứng dụng yêu cầu phạm vi lớn. LoRaTM cũng cung cấp các lợi thế đáng kể trong cả việc chặn và chọn lọc so với các kỹ thuật điều chế thông thường, giải quyết các vấn đề về phạm vi, chống nhiễu và tiêu thụ năng lượng so với các thiết kế truyền thống.
- Module này cũng hỗ trợ các chế độ FSK hiệu suất cao (G) cho các hệ thống bao gồm WMBus, IEEE802.15.4g. RFM95W cung cấp nhiễu pha đặc biệt, độ chọn lọc, độ tuyến tính của máy thu và IIP3 cho mức tiêu thụ hiện tại thấp hơn đáng kể so với các thiết bị cạnh tranh.

Thông số kĩ thuật:

- Tương thích chuẩn LoRaTM
- 168 dB maximum link budget.
- Công suất phát RF +20 dBm - 100 mW ổn định khi điện áp cung cấp không ổn định.
- Bộ khuých đại công suất (PA)+14 dBm hiệu suất cao.
- Tốc độ truyền tải dữ liệu có thể lên tới 300 kbps.
- Độ nhạy cao: -148 dBm.
- Bullet-proof front end: IIP3 = -12.5 dBm.
- Excellent blocking immunity.
- Low RX current of 10.3 mA, 200 nA register retention.
- Fully integrated synthesizer with a resolution of 61 Hz.
- FSK, GFSK, MSK, GMSK, LoRaTM and OOK modulation.
- Built-in bit synchronizer for clock recovery.
- Preamble detection.
- 127 dB Dynamic Range RSSI.
- Automatic RF Sense and CAD with ultra-fast AFC.
- Tích hợp bộ đóng gói dữ liệu lên tới 256 byte với kiểm tra dữ liệu CRC.
- Tích hợp cảm biến nhiệt độ và điện áp thấp.
- Kích thước : 16*16mm

2.5.3 Định vị GPS Neo 7M

Giới thiệu:



Hình 2. 11: Module GPS Neo 7M

Thông số kĩ thuật:

- Điện áp hoạt động: 3.3V đến 5VDC.
- Cổng kết nối giao tiếp: UART TTL, 9600bps.
- Đèn báo tín hiệu LED
- Pin-out cho nguồn và cổng kết nối
    - VCC: 3.3 đến 5VDC
    - RX: Dữ liệu nhận vào (đầu vào cho module GPS)

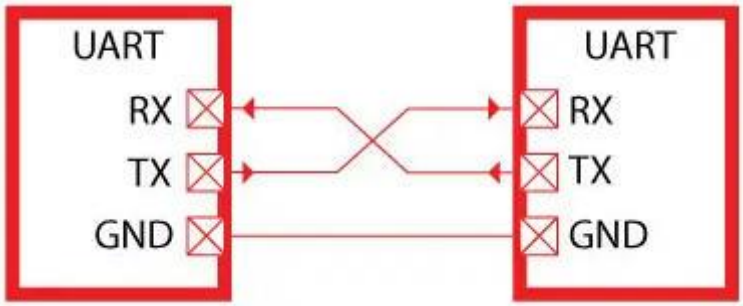
- TX: Dữ liệu truyền ra ngoài (ra từ module GPS)
- GND: Ground (mặt đất nguồn và logic)

- Kích thước: 36 x 26 mm.
- Trọng lượng: 22g.

2.6 Giao tiếp UART

Giới thiệu:

- UART (Universal Asynchronous Receiver-Transmitter – Bộ truyền nhận dữ liệu nối tiếp bất đồng bộ) là một trong những giao thức truyền thông giữa thiết bị với thiết bị được sử dụng nhiều nhất. Bạn có thể thấy giao tiếp UART được sử dụng nhiều trong các ứng dụng để giao tiếp với các module như: Wifi, Bluetooth, Xbee, module đầu đọc thẻ RFID với Raspberry Pi, Arduino hoặc vi điều khiển khác. Đây cũng là chuẩn giao tiếp thông dụng và phổ biến trong công nghiệp từ trước đến nay.



Hình 2. 12: Sơ đồ ghép nối

Hai đường dây mà mỗi thiết bị UART sử dụng để truyền dữ liệu đó là:

- **Transmitter (Tx)**
- **Receiver (Rx)**

UART truyền dữ liệu không đồng bộ, có nghĩa là không có tín hiệu đồng hồ để đồng bộ hóa đầu ra của các bit từ UART truyền đến việc lấy mẫu các bit bởi UART nhận. Thay vì tín hiệu đồng hồ, UART truyền thêm các bit start và stop vào gói dữ liệu được chuyển. Các bit này xác định điểm bắt đầu và điểm kết thúc của gói dữ liệu để UART nhận biết khi nào bắt đầu đọc các bit.

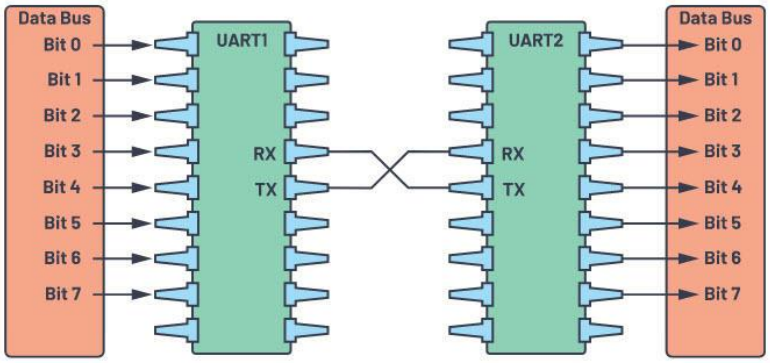
Khi UART nhận phát hiện một bit start, nó bắt đầu đọc các bit đến ở một tần số cụ thể được gọi là tốc độ truyền (baud rate). Tốc độ truyền là thước đo tốc độ truyền dữ liệu, được biểu thị bằng bit trên giây (bps – bit per second). Cả hai UART đều phải hoạt động ở cùng một tốc độ truyền.

Cả hai UART cũng phải được cấu hình để truyền và nhận cùng một cấu trúc gói dữ liệu.

Số lượng dây sử dụng	2
Tốc độ truyền	9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600, 1000000, 1500000
Phương pháp truyền	Bất đồng bộ
Truyền nối tiếp hay song song	Nối tiếp
Số lượng thiết bị chủ tối đa	1
Số lượng thiết bị tớ tối đa	1

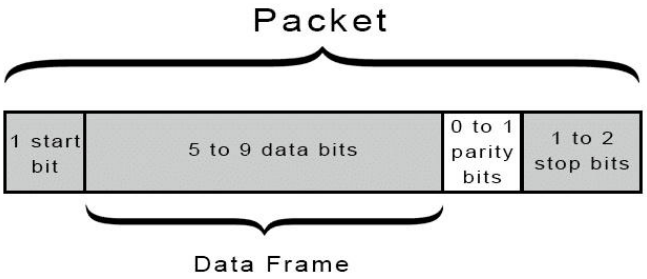
Bảng 2. 13: Cấu trúc gói dữ liệu UART

UART sẽ truyền dữ liệu nhận được từ một bus dữ liệu (Data Bus). Bus dữ liệu được sử dụng để gửi dữ liệu đến UART bởi một thiết bị khác như CPU, bộ nhớ hoặc vi điều khiển. Dữ liệu được chuyển từ bus dữ liệu đến UART truyền ở dạng song song. Sau khi UART truyền nhận dữ liệu song song từ bus dữ liệu, nó sẽ thêm một bit start, một bit chẵn lẻ và một bit stop, tạo ra gói dữ liệu. Tiếp theo, gói dữ liệu được xuất ra nối tiếp từng bit tại chân Tx. UART nhận đọc gói dữ liệu từng bit tại chân Rx của nó. UART nhận sau đó chuyển đổi dữ liệu trở lại dạng song song và loại bỏ bit start, bit chẵn lẻ và bit stop. Cuối cùng, UART nhận chuyển gói dữ liệu song song với bus dữ liệu ở đầu nhận.



Hình 2. 14: Đấu nối giữa hai thiết bị truyền nhận UART

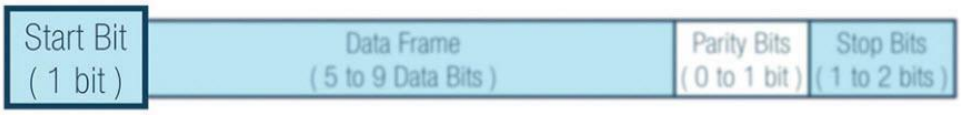
Dữ liệu truyền qua UART được tập hợp thành gói (packet). Mỗi gói chứa 1 bit start, 5 đến 9 bit dữ liệu (tùy thuộc vào UART), một bit chẵn lẻ (parity bit) tùy chọn và 1 hoặc 2 bit stop.



Hình 2. 85: Cấu trúc khung truyền

Start bit (Bit bắt đầu)

Đường truyền dữ liệu UART thường được giữ ở mức điện áp cao khi nó không truyền dữ liệu. Để bắt đầu truyền dữ liệu, UART truyền sẽ kéo đường truyền từ mức cao xuống mức thấp trong một chu kỳ đồng hồ. Khi UART nhận phát hiện sự chuyển đổi điện áp cao xuống thấp, nó bắt đầu đọc các bit trong khung dữ liệu ở tần số của tốc độ truyền.



Hình 2. 16: cấu trúc khung truyền

Data Frame (Khung dữ liệu)

Khung dữ liệu chứa dữ liệu thực tế đang được truyền. Nó có thể dài từ 5 bit đến 8 bit nếu sử dụng bit chẵn lẻ. Nếu không sử dụng bit chẵn lẻ, khung dữ liệu có thể dài 9 bit. Trong hầu hết các trường hợp, dữ liệu được truyền với bit có trọng số bé nhất (LSB – Least Significant Bit) trước tiên.



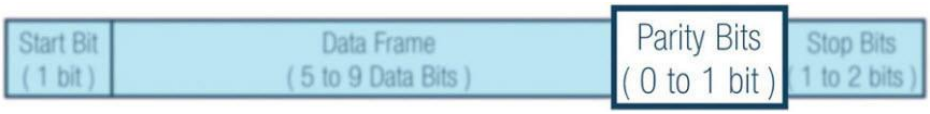
Hình 2. 17: Cấu trúc khung truyền

Parity Bit (Bit chẵn lẻ)

Tính chẵn lẻ mô tả tính chẵn hoặc lẻ của một số. Bit chẵn lẻ là một cách để UART nhận cho biết liệu có bất kỳ dữ liệu nào đã thay đổi trong quá trình truyền hay không. Bit có thể bị thay đổi bởi bức xạ điện từ, tốc độ truyền không khớp hoặc truyền dữ liệu đường dài.

Sau khi UART nhận đọc khung dữ liệu, nó sẽ đếm số bit có giá trị là 1 và kiểm tra xem tổng số là số chẵn hay lẻ. Nếu bit parity là 0 (even parity – parity chẵn), thì tổng số bit 1 trong khung dữ liệu phải luôn là một số chẵn. Nếu bit parity là 1 (odd parity – parity lẻ) thì số tổng bit 1 trong khung dữ liệu là một số lẻ.

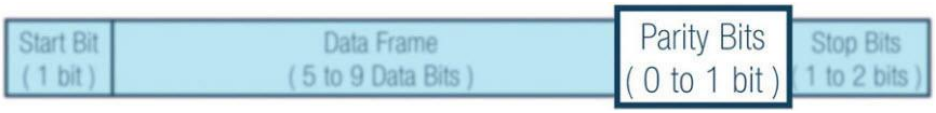
Khi bit chẵn lẻ khớp với dữ liệu, UART biết rằng quá trình truyền không có lỗi. Nhưng nếu bit chẵn lẻ là 0 và tổng là số lẻ, hoặc bit chẵn lẻ là 1 và tổng số là chẵn, thì UART biết rằng các bit trong khung dữ liệu đã thay đổi.



Hình 2. 98: Cấu trúc khung truyền

Stop Bit (Bit kết thúc)

Để báo hiệu sự kết thúc của gói dữ liệu, UART gửi sẽ điều khiển đường truyền dữ liệu từ điện áp thấp đến điện áp cao trong ít nhất hai khoảng thời gian bit.



Hình 2. 19: Cấu trúc khung truyền

2.7 Giao tiếp SPI

Giới thiệu:

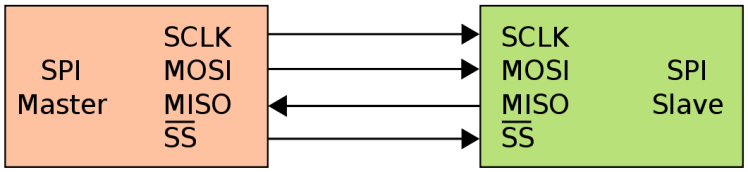
Chuẩn giao tiếp SPI được hãng Motorola đề xuất, ra đời vào giữa những năm 1980 để giải quyết về nhu cầu về việc giao tiếp giữa các thiết bị với thông lượng nhanh hơn giao thức I2C hiện có.

SPI (Serial Peripheral Interface) – Hay còn được gọi là Four wire – Chuẩn giao tiếp 4 dây. Kênh truyền nối tiếp song công (full duplex – có 2 đường truyền dữ liệu riêng biệt, một đường truyền và một đường nhận, cho phép dữ liệu truyền nhận theo 2 hướng cùng lúc).

SPI là một giao thức kiểu Master – Slave. Master là thiết bị điều khiển (thường là vi điều khiển), còn Slave (thường là các cảm biến, màn hình LCD hoặc chip nhớ) sẽ nhận lệnh từ Master.

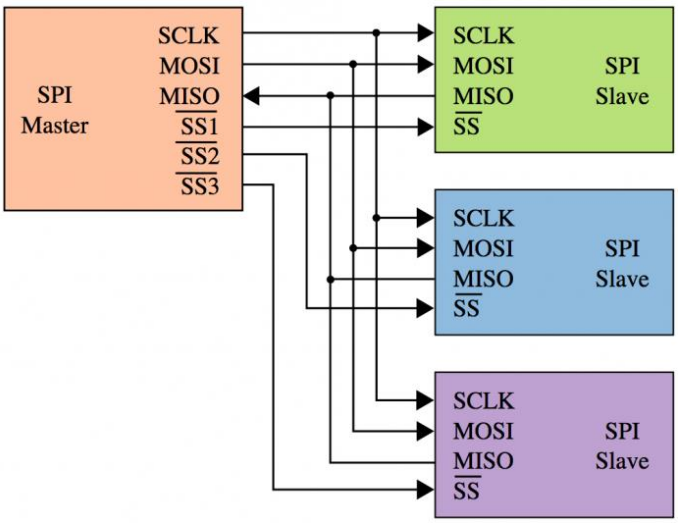
Một hệ thống SPI chỉ gồm một thiết bị Master, nhưng lại có thể có một hoặc nhiều thiết bị Slave.

Khối SPI của thiết bị Master sẽ kết nối với khối SPI của thiết bị Slave qua 4 chân tín hiệu:



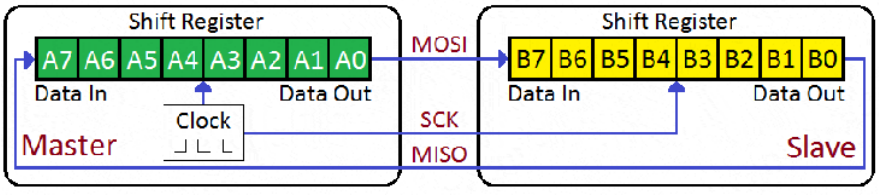
Hình 2.20: Sơ đồ kết nối SPI Master và SPI Slave

- MOSI (Master Output/Slave Input) – Master gửi dữ liệu đến Slave.
- MISO (Master Input/Slave Output) – Slave gửi dữ liệu cho Master.
- SCLK (Serial Clock) – xung giữ nhịp Clock.
- SS/CS (Slave Select/Chip Select) – Master lựa chọn Slave để giao tiếp. Cụ thể trong trường hợp hệ thống gồm một Master giao tiếp với nhiều Slave, Master điều khiển tín hiệu mức logic trên chân SS/CS của Slave nào xuống mức LOW (mức 0), nghĩa là Master đang giao tiếp với Slave đó.



Hình 2.21: Sơ đồ ghép nối master với các slave

Cơ chế hoạt động:



Hình 2. 10: Cơ chế truyền dữ liệu

Mỗi thiết bị Master và Slave đều có một thanh ghi dịch 8 bits (Shift Register), một bộ tạo xung nhịp (Clock Generator).

Khi Master truyền dữ liệu, Master truyền đi 8 bits dữ liệu vào thanh ghi dịch của nó, sau đó 8 bits dữ liệu được truyền theo đường tín hiệu MOSI sang thiết bị Slave. Tương tự khi Slave truyền dữ liệu, các bits trên thanh ghi dịch của Slave truyền đến Master thông qua đường tín hiệu MISO.

Bằng cách này, dữ liệu của hai thanh ghi sẽ được trao đổi với nhau. Việc đọc và ghi dữ liệu vào Slave diễn ra cùng một lúc nên tốc độ trao đổi dữ liệu diễn ra rất nhanh. Do đó, giao thức SPI là một giao thức rất có hiệu quả.

2.8 Datacake

Giới thiệu

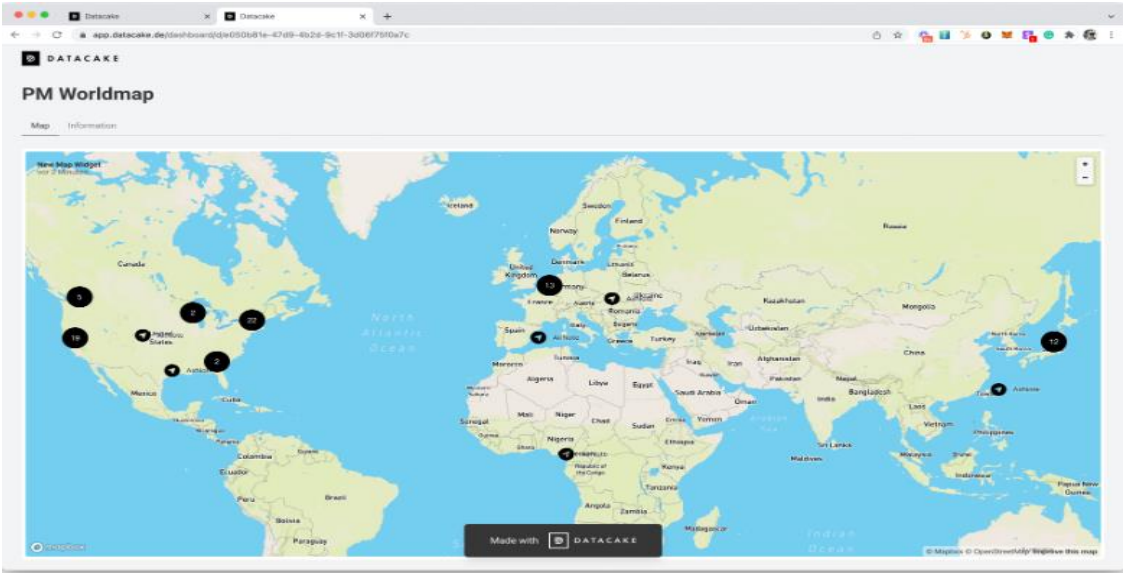
Hình 2. 22: Datacake iots

DataCake là một nền tảng IOT đa mục đích, mã thấp, không yêu cầu kĩ luật lập trình và thời gian tối thiểu để tạo các ứng IOT tùy chỉnh có thể đưa vào giải pháp IOT nhãn trắng chỉ bằng một nút nhấn.

Datacake cho phép kết nối thiết bị thông qua giao thức IoT tiêu chuẩn công nghiệp – MQTT(API)…

Các tính năng của DataCake:

- Hiển thị bản đồ: Datacake Dashboard Editor có một tiện ích con để hiển thị một hoặc nhiều vị trí, bao gồm tùy chọn hiển thị các tuyến đường lịch sử và khả năng tương tác.



Hình 2.23 Widget Map Datacake

- Thu thập dữ liệu từ xa: Datacake sẽ hỗ trợ bạn thu thập và lưu trữ dữ liệu từ xa theo cách đáng tin cậy.
- Tính năng members: Thành viên là Tài khoản Người dùng có quyền truy cập vào không gian làm việc nhất định. Bạn có thể thêm nhiều thành viên vào không gian làm việc của mình và xác định các quyền và vai trò cũng như các thiết bị mà người dùng được phép xem, kiểm soát, thay đổi.

2.9 Kết thúc chương

Chương 2 đã nêu ra được những cơ sở lý thuyết về các thiết bị được sử dụng trong hệ thống, từ đó chúng ta có thể rút ra được các yêu cầu cần thiết để thực hiện việc thiết kế phần cứng của đề tài xây dựng hệ thống giám sát này. Ngoài ra, chương 2 còn đưa ra những giới thiệu sơ bộ về một số linh kiện điện tử, những phần mềm, nền tảng có thể sử dụng được, đưa ra những ưu, nhược điểm của từng linh kiện, phần mềm để từ đó chúng ta có thể có được sự lựa chọn đúng đắn khi thiết kế, tính toán phần cứng, phần mềm ở Chương 3.

CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ HỆ THỐNG VÀ KẾT QUẢ, ĐÁNH GIÁ

3.1 Giới thiệu chương

Từ những thông tin rút ra được từ Chương 2, Chương 3 sẽ trình bày những yêu cầu đặt ra của hệ thống, từ những yêu cầu đó chúng ta sẽ xây dựng nên sơ đồ khối chức năng từng phần, rồi lựa chọn linh kiện phù hợp với từng khối chức năng. Sau khi đã chọn được những linh kiện chính chúng ta sẽ lựa chọn các sơ đồ mạch phù hợp của từng khối rồi sẽ phân tích tính toán thiết kế từng phần, mô phỏng mạch, vẽ mạch PCB và hoàn thiện mạch trong thực tế. Và cuối cùng là kết quả và đánh giá

3.2 Yêu cầu đặt ra cho hệ thống

Từ những thông tin của Chương 1 và Chương 2, chúng ta có những yêu cầu đặt ra cho hệ thống giám sát, thu nhận toạ độ, khoảng cách người bị nạn tới trạm cứu hộ như sau:

Hệ thống sẽ bao gồm ba phần chính:

- LoRaGateway
- End node
-Web App điều kiển, giám sát

3.3 Tính toán thiết kế phần cứng end node

3.3.1 Yêu cầu thiết kế và sơ đồ khối

Yêu cầu thiết kế:

Giám sát toạ độ ví trí, độ cao sô với mực nước biển, khoảng cách của khách du dịch so với trạm cứu hộ.

End node phải đạt các yêu cầu sau:

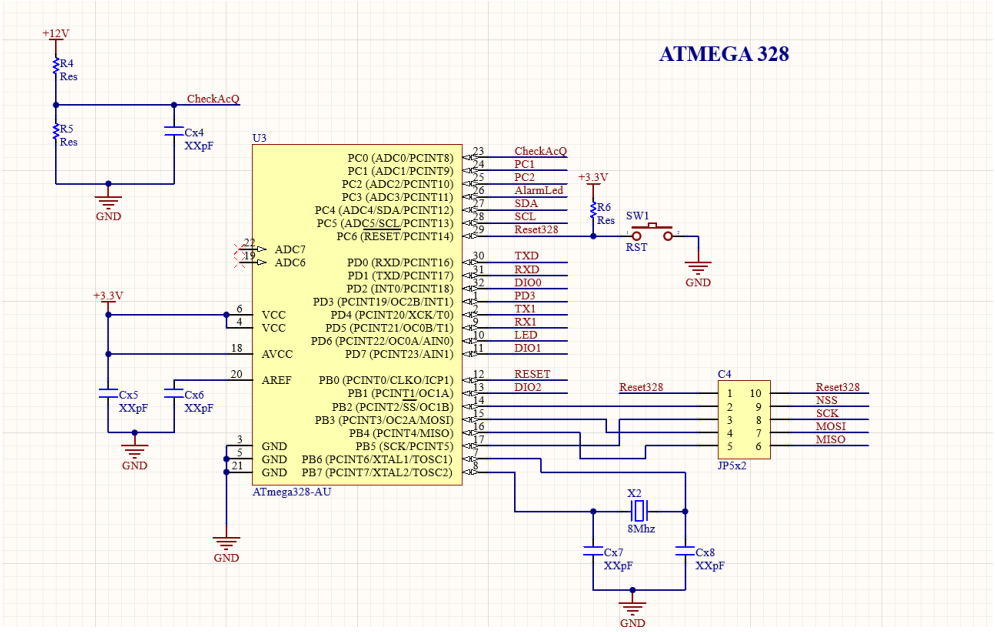
- Đọc chính xác giá trị cảm biến.
- Kết nối ổn định với gateway để truyền dữ liệu về server.
- Tiết kiệm năng lượng ở mức thấp nhất.
- Chịu được các điều kiện dưới nước cũng như ngoài trời

3.3.2 Tính toán thiết kế

3.3.2.1 MCU và module LoRa

a)Tính toán MCU

❖ Sơ đồ nguyên lý của MCU:



Hình 3.1: Sơ đồ nguyên lý MCU

Đối với đề tài này, chúng ta cần 1 MCU có khả năng tiết kiệm năng lượng.

❖ Phân tích tính toán thiết kế:

- Để vi điều khiển hoạt động bình thường điện áp hoạt động nằm trong khoảng từ 3.3V – 5V. Do đó chúng ta thiết kế nguồn cấp cho vi điều khiển hoạt động là 3.3V ở V_{CC} chân số 6, 4, 18, 20 và GND ở chân số 3, 5, 21. Ngoài ra để lọc nhiễu các tín hiệu cao tần chúng ta cần sử dụng thêm một tụ có điện dung từ 10nF – 100nF. Chúng ta chọn Cx4, Cx5, Cx6 là tụ 32nF (loại tụ 104).
- Tinh thể X2, tụ điện C6 và C7 được sử dụng cho mạch dao động đồng hồ. X2 là một tinh thể thạch anh với tần số 8MHz, và Cx7 và Cx8 là tụ gốm nguyên khối hoặc tần số cao với công suất 22pF. Nếu X2 sử dụng tinh thể gốm chi phí thấp, dung tích Cx7 và Cx8 phải được nhà sản xuất tinh thể khuyên dùng, thường là 32pF.(loại tụ 104)

- Theo datasheet của Atemega328p, MCU được reset bằng cách giữ chân RST ở mức thấp $\leq 0.6V$ trong khoảng thời gian tối thiểu 200µs, sau đó chuyển về mức cao. RST có thể tác động bằng tay hoặc tác động khi cấp nguồn bằng cách sử dụng mạch RC như hình trên. Chúng ta thiết kế mạch như sau:
- Phương trình nạp cho tụ C1 là:
-

$$v_c(t) = [v_c(\infty) - v_c(0)]\left(1 - e^{-\frac{t}{\tau}}\right) + v_c(0) \quad (1)$$

Với $v_c(\infty) = v_{cc} = 3.3V$ chính là điện áp nguồn của mạch.

$v(0) = 0V$ là điện áp ở thời điểm ban đầu.

$$\Rightarrow v_c(t) = v_{cc}\left(1 - e^{-\frac{t}{\tau}}\right) \quad (2)$$

Chọn thời gian reset t = t_{RST} = 2ms, để vi điều khiển reset cần giữ mức điện áp mức logic 0 = 0.6V trong 2ms:

$$\Rightarrow 0.6 = 3.3\left(1 - e^{-\frac{2\times10^{-3}}{\tau}}\right) \quad (3)$$

$$\Leftrightarrow e^{-\frac{2\times10^{-3}}{\tau}} = 0.818 \quad (4)$$

$$\Leftrightarrow \tau = -\frac{2\times10^{-3}}{\ln(0.818)} \approx 9.95\times10^{-3} \quad (5)$$

Mà thời hằng τ chính là tích của hai thành phần R và C mắc song song với dây tín hiệu RST, chính là R_6 và C_{12} theo sơ đồ mạch trên:

$$\tau = R_6 C_{12} \quad (6)$$

Chọn C_1 = 0.1uF => R_2 = 9.95kΩ.

Chúng ta lựa chọn điện trở sai số nên sẽ chọn R_6 = 10kΩ.

Chọn R4:R5= 5:1
Chọn R4=20kΩ => R5=100kΩ (7)

b) Module LoRa

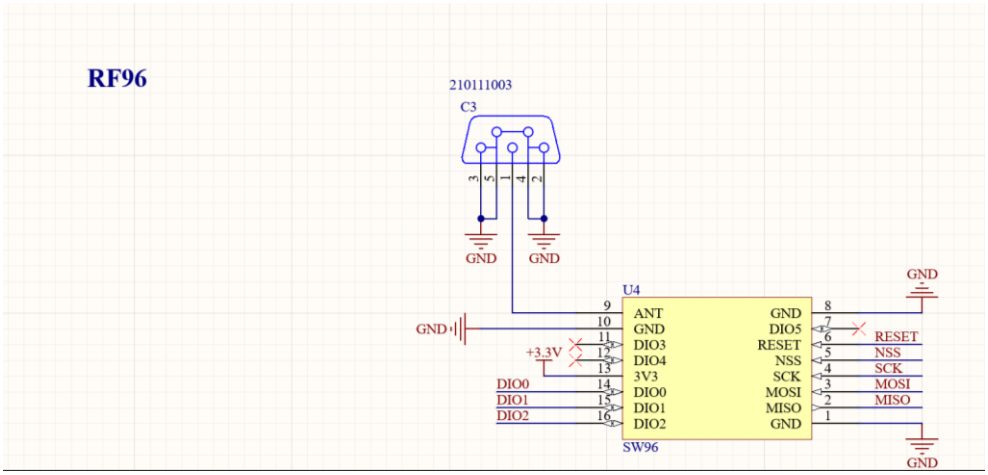
❖ Yêu cầu thiết kế:

Module LoRa phải đáp ứng được băng tần 923MHz do Cục tần số quy định tại Việt Nam.

Giao tiếp với MCU bằng giao tiếp SPI

Từ đó ta chọn module LoRa RF96W

❖ Sơ đồ nguyên lý:



Hình 3. 2: Sơ đồ module LoRa

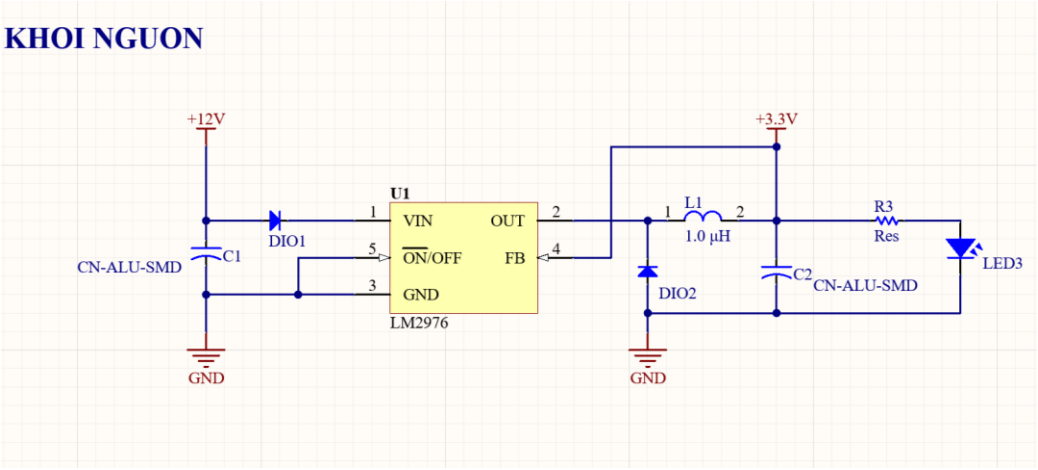
❖ Nguyên lý hoạt động

Module LoRa hoạt động ở băng tần 923MHz, và kiểu điều chế song ở module này là "Chirp Spread Spectrum" (CSS) như đã đề cập ở chương 2.

Điện áp 3,3 V được cấp từ khối nguồn.

3.3.2.2 Thiết kế khối nguồn

❖ Sơ đồ nguyên lý khối nguồn:



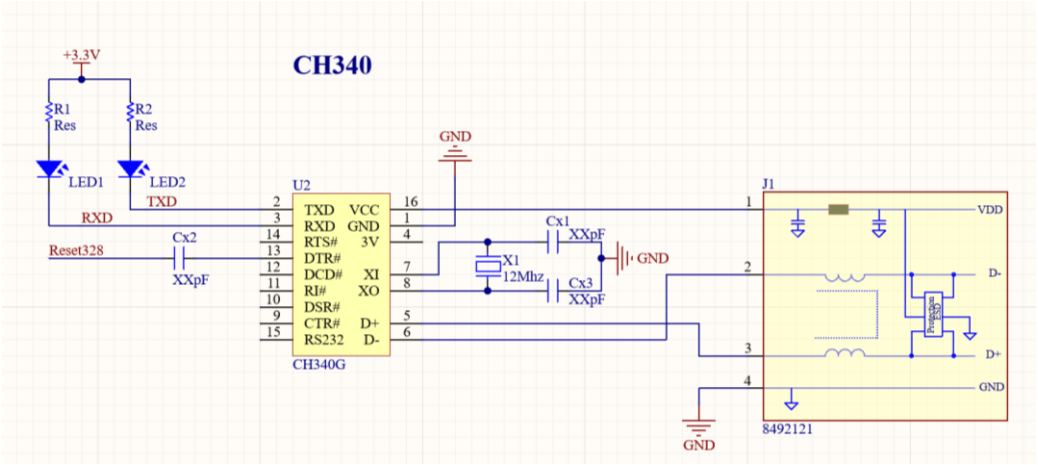
Hình3.3 Khối nguồn

❖ Tính toán:

- Trên thị trường có nhiều IC ổn áp 5V như : 7805, AMS1117 5V, LM2596,... Trong trường hợp này ta sử dụng IC LM2576 vì phổ biến và độ ổn định cao.
- Tụ C1,C2: Tụ san phẳng điện áp đầu và lọc bỏ các tín hiệu xung cao tần vào giúp bộ chuyển đổi hoạt động ổn định. Chọn tụ C1, C2 là tụ nhôm 220uF,25V đặt gần nguồn 12VDC để ổn định điện áp đầu vào và tụ C2 tạo thành mạch lọc LC.
- DIO2: Diode ghim áp đầu ra. Diode có dòng điện định mức >1.2 lần dòng tải tối đa, định mức điện áp ngược của diode bằng 1.25 lần điện áp vào tối đa => Chọn Diode Schottky 30V 3.0A 1N5824 (SS34).
- DIO1: Diode 1N4007.
- L1: Cuộn cảm chuyển hóa năng lượng. Cuộn cảm có giá trị dòng tối đa >3.0A và điện cảm >27uH => Chọn cuộn cảm CD127 470uH 3A.
- Chọn R3= 1kΩ.

3.3.2.3 Thiết CH340

❖ Sơ đồ nguyên lý:



Hình 3. 4: Mạch chuyển đổi CH340

❖ Nguyên lý hoạt động:

- Chân TXD của CH340 phải được kết nối với chân RXD của MCU và chân RXD phải được kết nối với chân TXD của MCU.
- Ngoài ra, diode được kết nối với chân TXD và điện trở được kết nối với RXD để ngăn USB cung cấp năng lượng cho chip mục tiêu.
- Chip CH340 có điện trở kéo lên USB tích hợp. Các chân D + và D- của CH340 phải được kết nối trực tiếp với bus USB
- Khi sử dụng điện áp hoạt động 3,3V, chân 3V của chip CH340 phải được kết nối với chân VCC và nguồn điện 3,3V bên ngoài phải được nhập và các mạch khác được kết nối với chip CH340 không được vượt quá 3,3V.
- Hỗ trợ baurate từ 2400bps đến 115200bps.

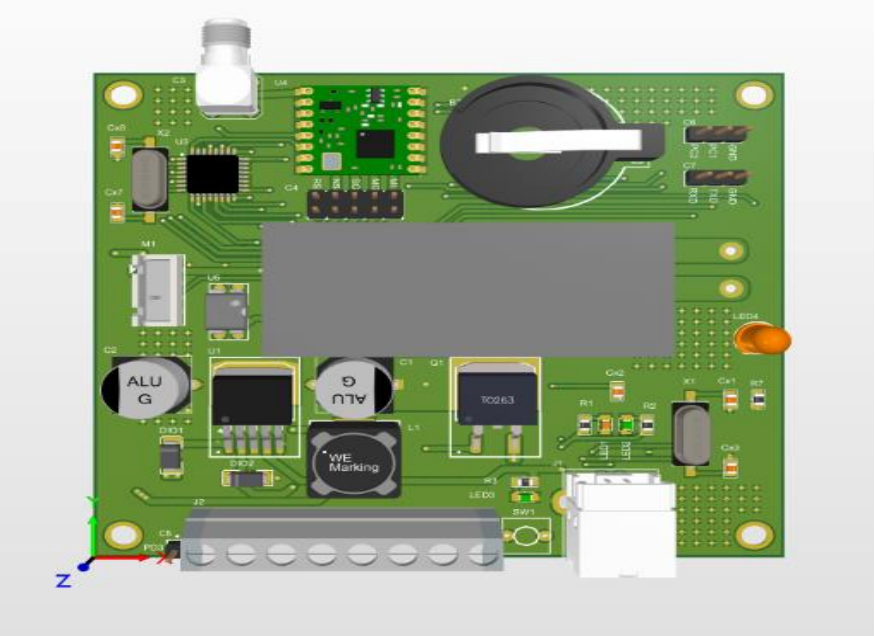
❖ Tính toán:

- Tinh thể X1, tụ điện Cx1 và Cx3 được sử dụng cho mạch dao động đồng hồ. X1 là một tinh thể thạch anh với tần số 12MHz, và Cx1 và Cx3 là tụ gốm nguyên khối hoặc tần số cao với công suất 22pF. Nếu X1 sử dụng tinh thể gốm chi phí thấp, dung tích Cx1 và Cx3 phải được nhà sản xuất tinh thể khuyên dùng, thường là 32pF.
- Ngoài ra để lọc nhiễu các tín hiệu cao tần và chọn thời gian reset cho CH340 chúng ta cần sử dụng thêm một tụ có điện dung từ 10nF – 100nF. Chúng ta chọn Cx2 là tụ 32nF (loại tụ 104).
- R1,R2 là trở hạn dòng; LED1, LED2 để báo hiệu TX à RX của CH340 và Atemega328p đã kết nối với nhau. Nên ta chọn R1=R2=1KΩ.

3.4 Layout mô hình mạch và sản phẩm thực tế

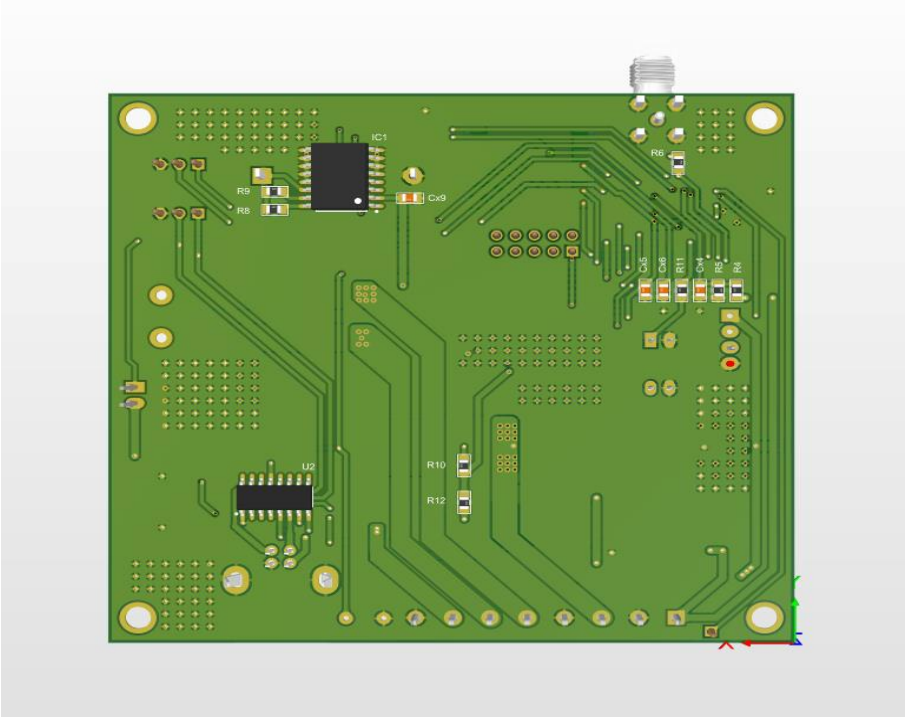
3.4.1 Mạch end node

Phần linh kiện được gắn chủ yếu ở mặt trên của mạch.



Hình 3. 5: Layout 3D mặt trên của end node

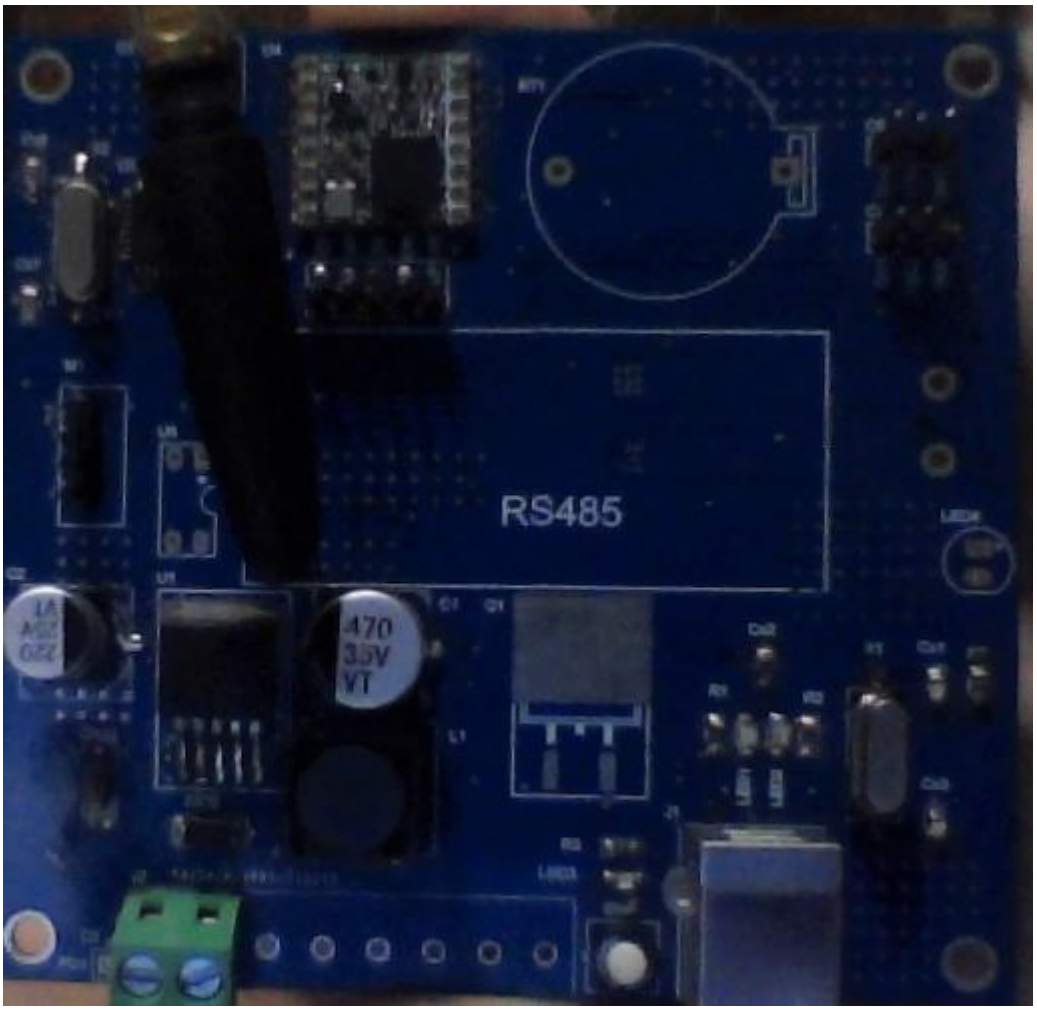
Phần ở giữa của mặt phía trên của mạch sẽ được dành để gắn GPS, vừa tạo thẩm mỹ cho mạch vừa giảm bớt đi độ dày, cộm của sản phẩm mạch.



Hình 3. 6: Layout 3D mặt dưới của end node

3.4.2 Sản phẩm mạch thực tế

Hình dưới đây là node cảm biến độ GPS



Hình 3. 7: Mạch node cảm biến GPS

3.5 Gateway

Gateway được sử dụng trong hệ thống là LPS8 LoRaWAN Gateway.



Hình 3. 8: LPS8 LoRaWAN Gateway

a) Giới thiệu

LPS8 là một LoRaWAN Gateway mã nguồn mở. Nó cho phép bạn kết nối mạng không dây LoRa với mạng IP qua WiFi hoặc Ethernet. Không dây LoRa cho phép người dùng gửi dữ liệu và đạt được phạm vi cực xa với tốc độ dữ liệu thấp. LPS8 sử dụng bộ chuyển tiếp gói Semtech và hoàn toàn tương thích với giao thức LoRaWAN. Nó bao gồm một bộ tập trung SX1308 LoRa, cung cấp 10 đường dẫn giải điều chế song song có thể lập trình được. LPS8 có các dải tần LoRaWAN tiêu chuẩn được cấu hình sẵn để sử dụng cho các quốc gia khác nhau, người dùng cũng có thể tùy chỉnh các dải tần để sử dụng trong mạng LoRa của riêng mình.

b) Thông số kỹ thuật

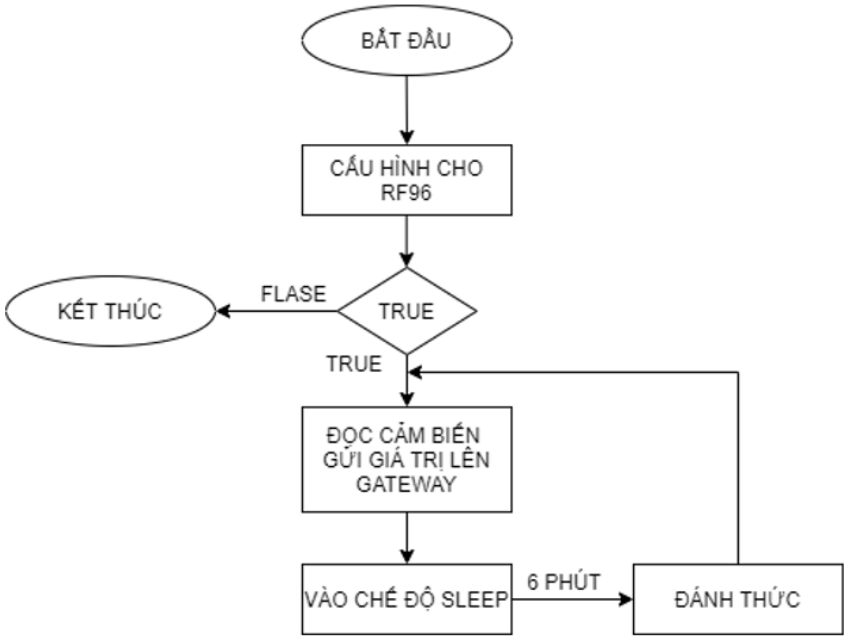
Các tính năng:

- Hệ thống mã nguồn mở OpenWrt
- Được quản lý bởi Web GUI, SSH qua mạng LAN hoặc WiFi
- Mô phỏng 49x bộ giải điều chế LoRa
- LoRaWAN Gateway 10 đường dẫn giải điều chế song song có thể lập trình.
- Thông số kỹ thuật: LoRa Thông số:
- 1 x SX1308 + 2 x 1257 LoRa Tranceiver

- Thông số chung: 1 cổng RJ45 10M / 100M
- 2.4G WiFi (802.11 bgn)
- Cổng máy chủ USB
- Nguồn điện vào qua USB Type-C: 5V, 2A
- Băng tần: US915, AU915, AS923, KR920

3.6 Thiết kế phần mềm

3.6.1 Thiết kế chương trình node cảm biến



Hình 3. 9: Thuật toán node cảm biến

Giải thích:

Sau khi khởi động node sẽ cấu hình cho module LoRa nếu sai thì dừng lại, nếu đúng thì tiếp tục đọc giá trị các cảm biến rồi gửi giá trị lên application server thông qua gateway. Sau đó node vào chế độ sleep, sau 6 phút sẻ được đánh thức và tiếp tục đọc giá trị cảm biến rồi gửi đi.

3.6.2 Hệ thống phần mềm

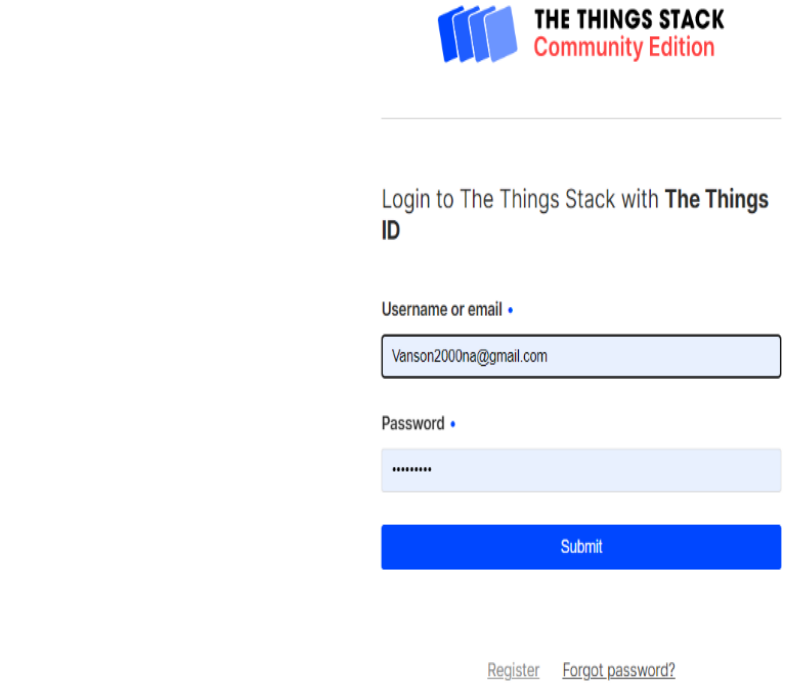
Hệ thống phần mềm gồm các thành phần chính:

- LoRaServer: Project sử dụng The Things NetWord, là một LoRaServer, nơi giải mã dữ liệu nhận được từ end node thông qua LoRaGateway và gửi dữ liệu xuống end node thông qua LoRaGateway.

Cách cài đặt:

❖ LoRaServer:

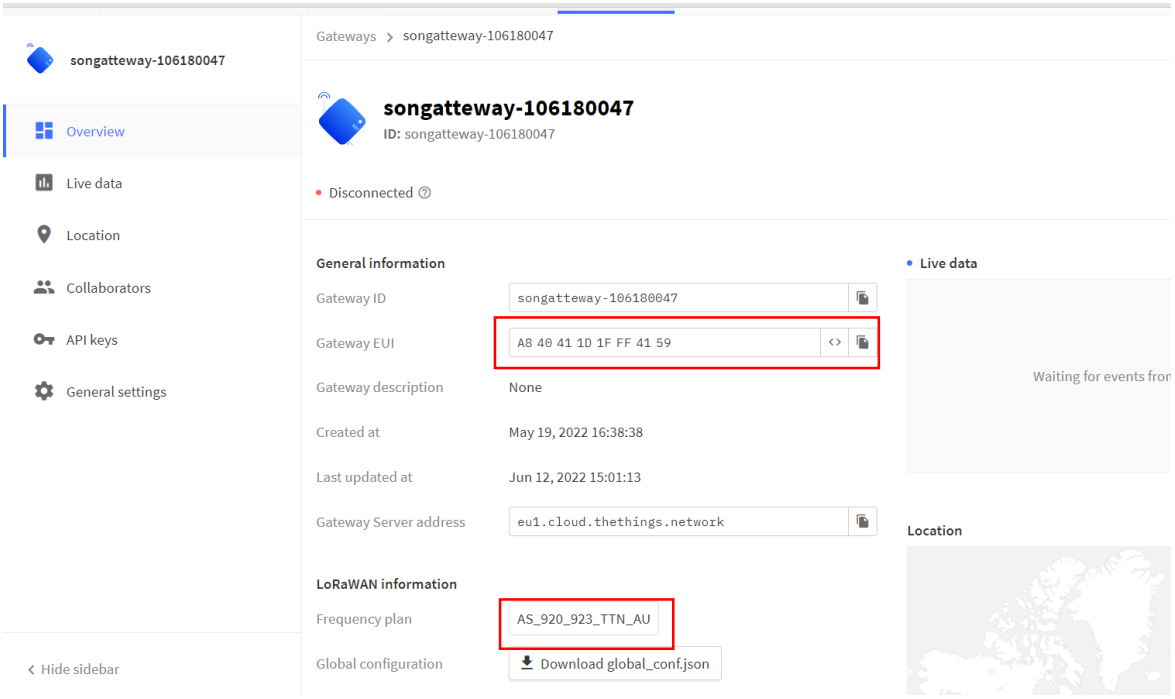
+ Đăng kí tài khoản trên The Things NetWork và mở Console để thực hiện dự án.



Hình 3. 10: Tạo tài khoản TTN

+ Thiết lập cài đặt tạo gateway:

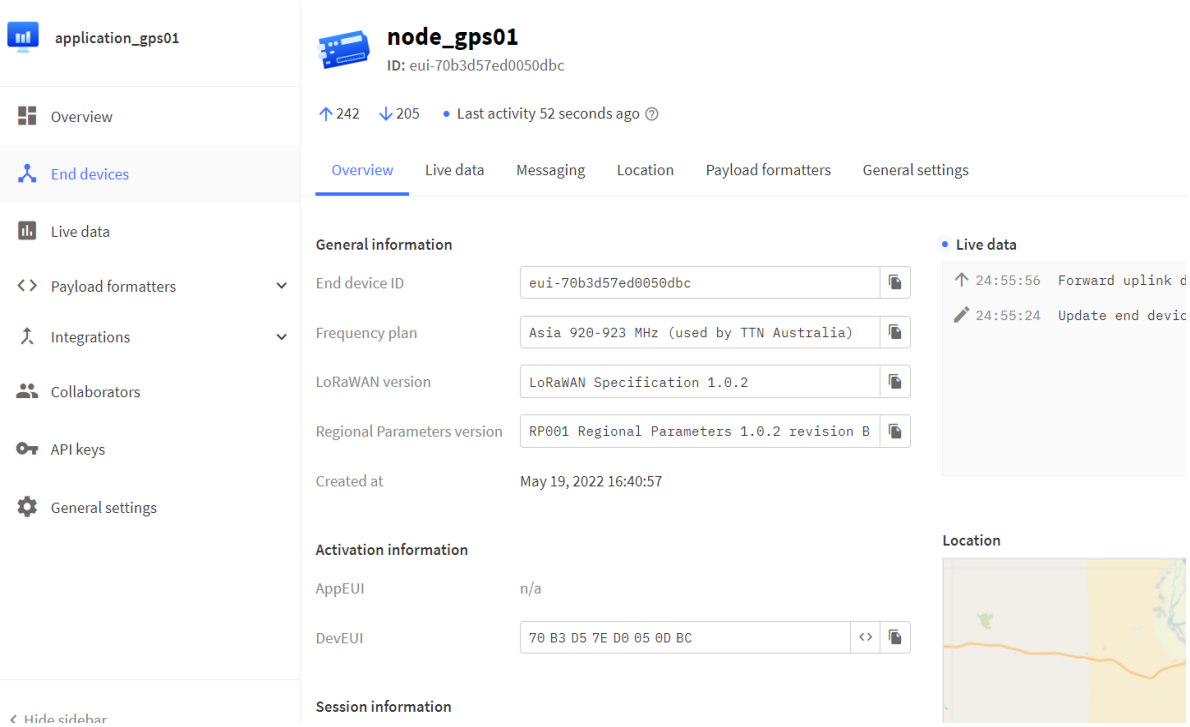
- Địa chỉ Gateway được tạo từ Gateway Dragino LSP8
- Tần số được sử dụng là: Asia 920 – 923 MHZ(used by TTN Australia)



Hình 3.11: Thiết lập Gateway

+ Thiết lập cài đặt cho từng end node(end divice).

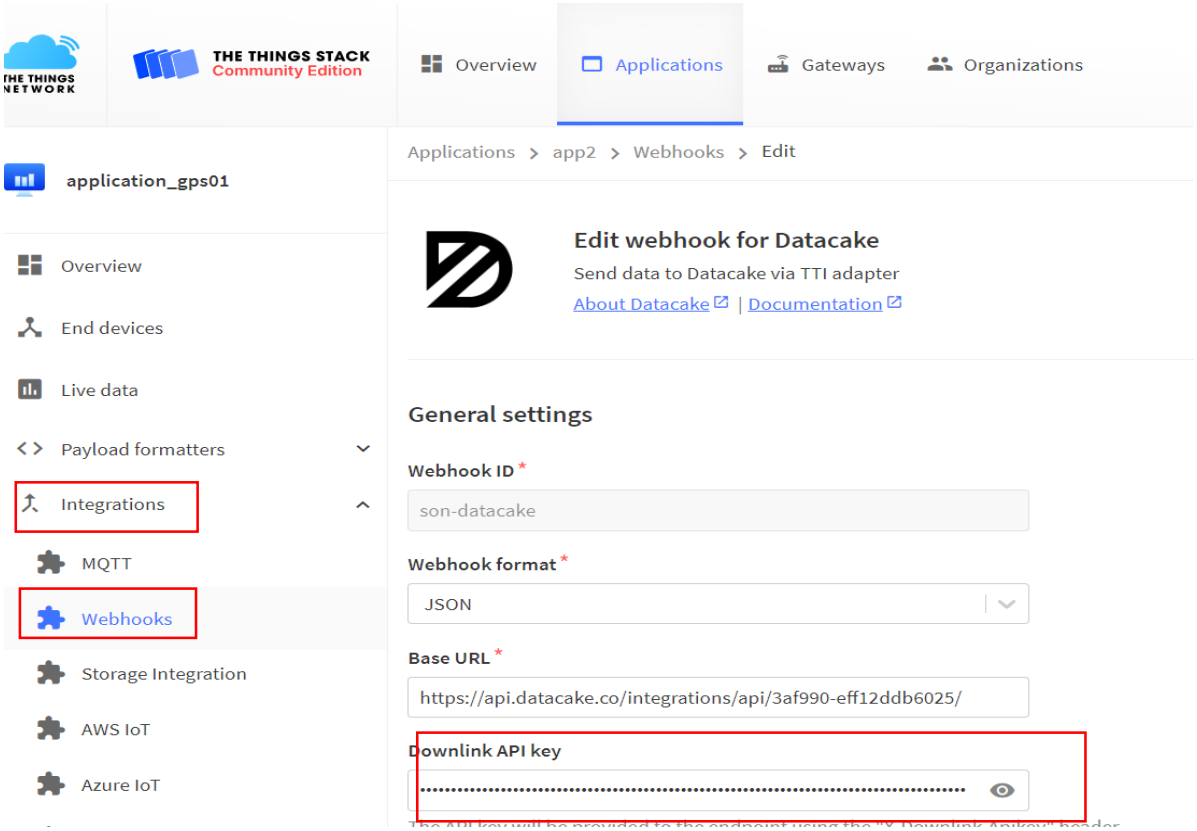
- Sử dụng chế độ ABP



Hình 3.12: End Node

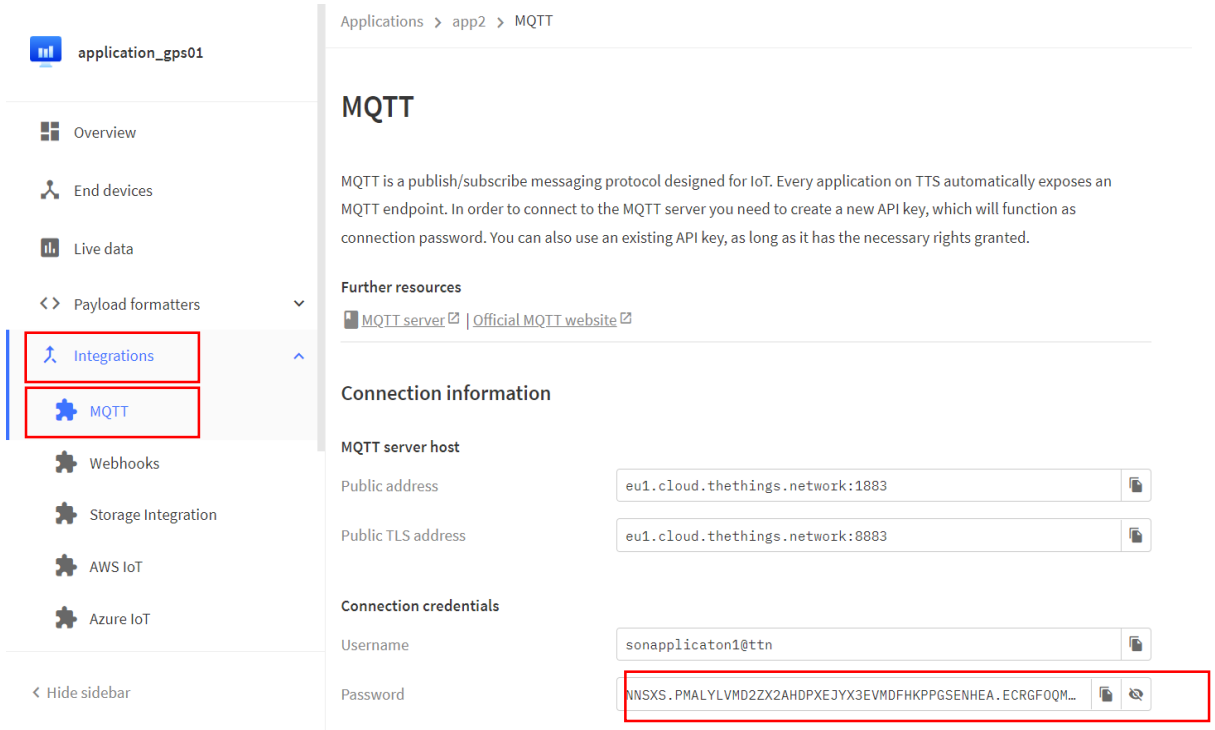
❖ Webhooks:

- Thiết lập cài đặt một wehook liên kết với The Things NetWork.



Hình 3.13: Thiết lập Webhooks

+ Thiết lập API liên kết với web app Datacake và Webhooks

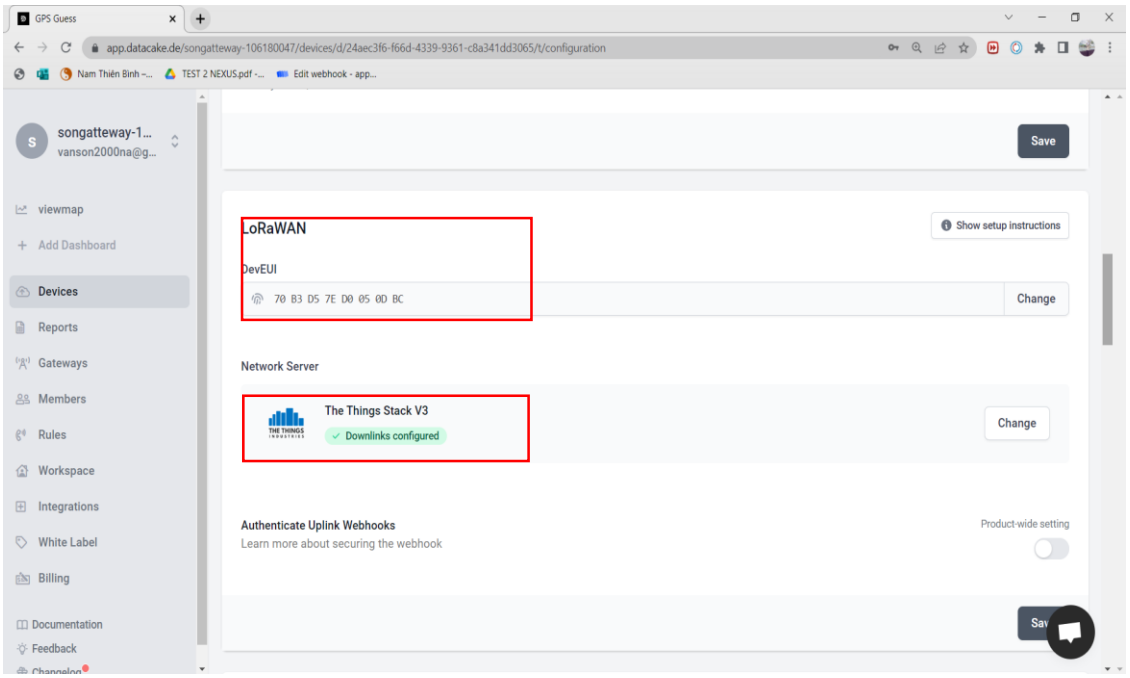


Hình 3.14: Thiết lập API từ MQTT

❖ Datacake:

+ Tạo tài khoản datacake.

+ Thiết lập cài đặt kết nối end node với datacake và webhooks

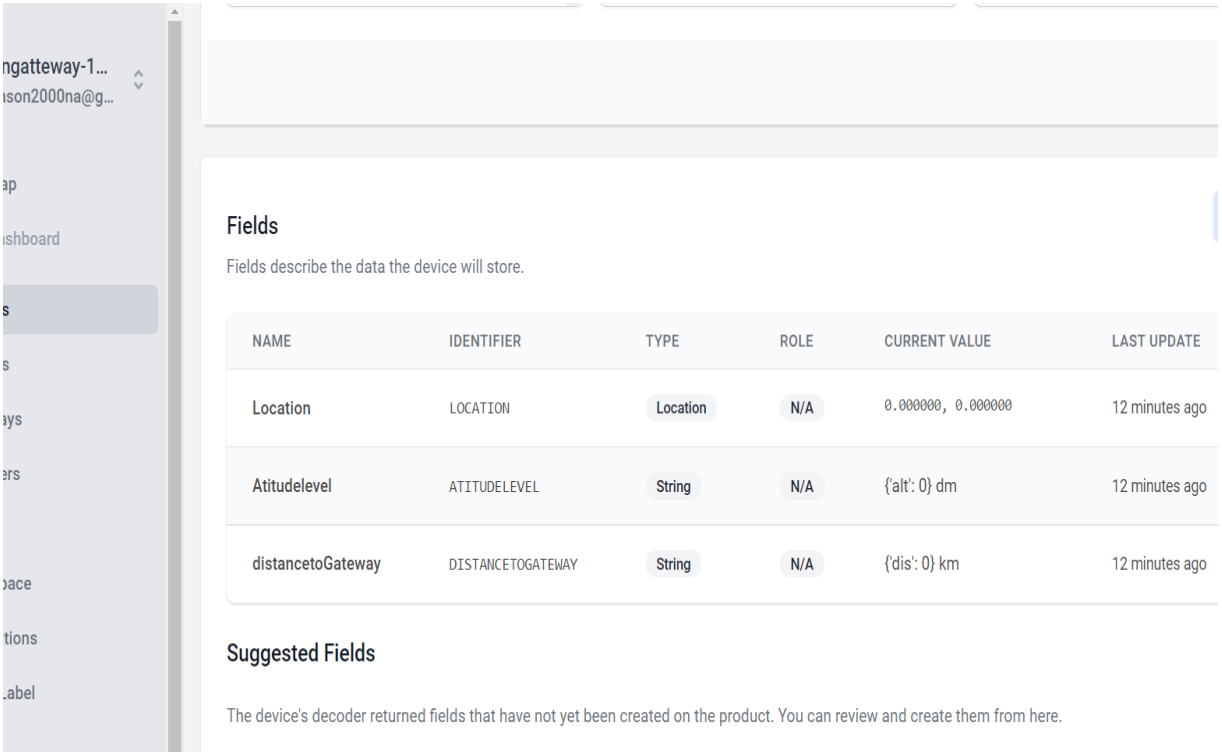


Hình 3.15: Kết nối end node với Webhook và datacake

+ Tạo Payload giải mã, fields và Maps để hiển thị kết quả hiển thị kết quả.



Hình 3.16: Payload Decoder

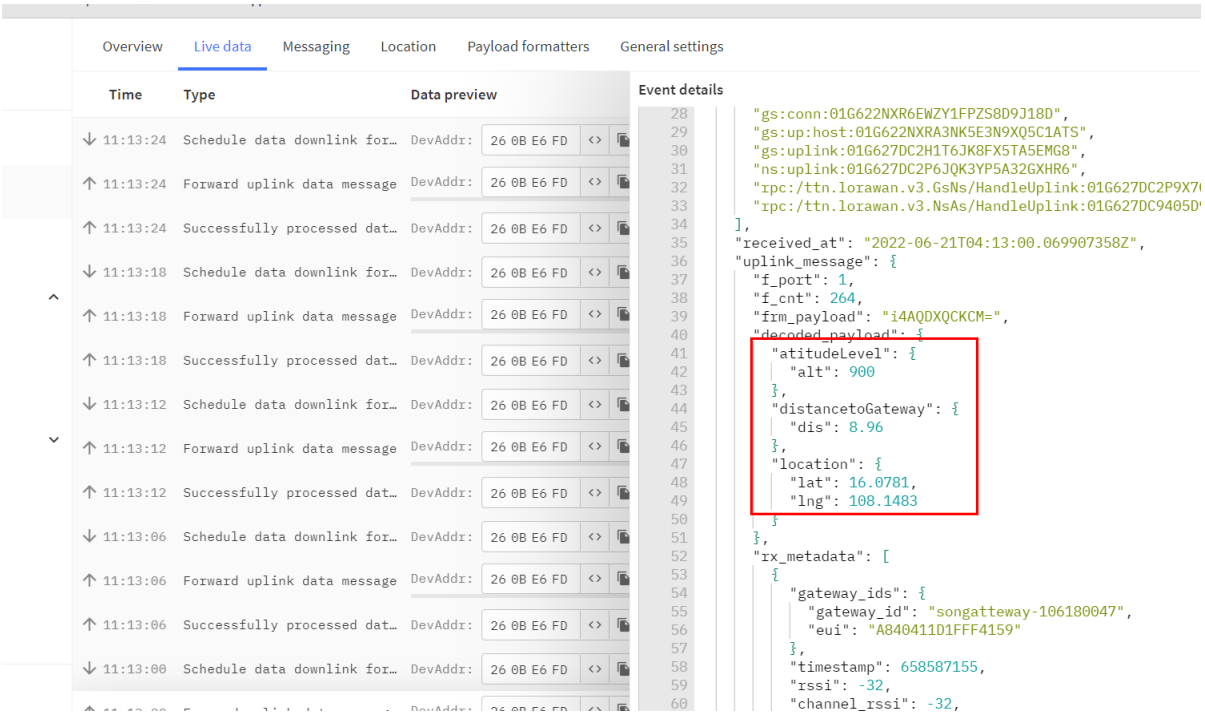


Hình 3.17: Fields

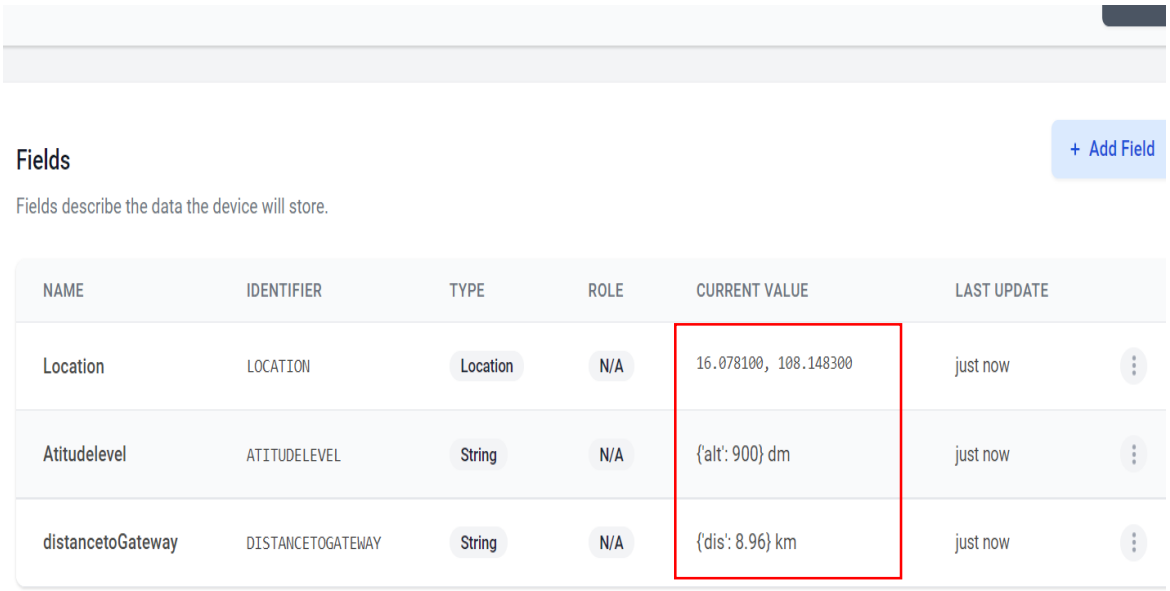
3.7 Kết quả Demo thực nghiệm

Đầu tiên chúng ta sẽ chọn vị trí là trạm cứu hộ cũng như là vị trí của Gateway có toạ độ: {16.072876404103216, 108.22643589725956}: địa chỉ: Cầu sông Hàn để xác định được khoảng cách.

Sau khi nạp code, Gps gửi dữ liệu lên server đến end divice:



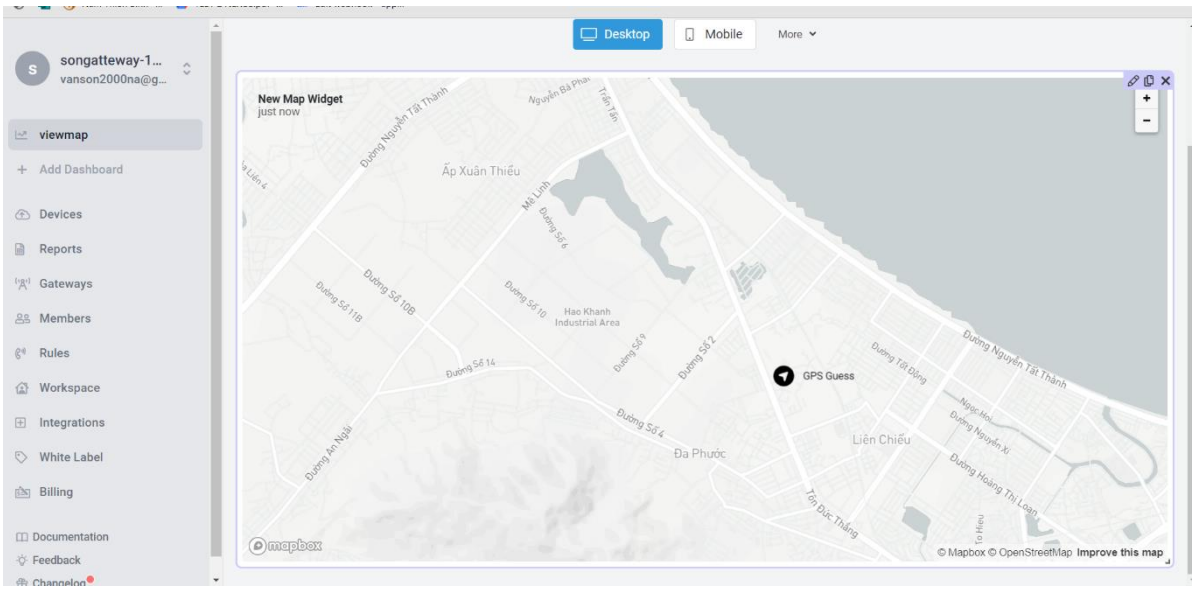
Thông qua API thì dữa liệu gửi qua webapp Datacake và được giải mã và hiển thị lên Fields:



Dự liệu thu nhận được:

- Toạ độ : **16.078100, 108.148200**
- Độ cao so mực nước biển: 900 dm.
- Khoảng cách: 8.96 Km.

Hiển thị toạ độ lên Widget Map:



Như vậy ta đã xác định được vị trí cần tìm

3.8 Đánh giá kết quả:

- Ưu và nhược điểm:

Ưu điểm:

- Độ chính xác của các cảm biến rất chính xác
- Hệ thống sử dụng công nghệ truyền dẫn LoRaWan nên khả năng phủ rộng, khoảng cách truyền dẫn xa với mức năng lượng thấp, bảo mật dữ liệu cao, khả năng tích hợp các node dễ dàng.
- Kích thước các mạch không quá lớn, thiết kế trên board mạch công nghiệp nên đáp ứng được tính thẩm mỹ cho người sử dụng.
- Node hoạt động với thời gian dài vì có khả năng tiết kiệm năng lượng.

Nhược điểm: Ngoài những ưu điểm kể trên thì hệ thống vẫn còn tồn tại một số hạn chế, đó là:

- Vì truyền dẫn bằng vô tuyến nên có thể bị nhiễu sóng từ môi trường hơn truyền bằng hữu tuyến.

3.9 Kết thúc chương.

Qua chương này ta đã tiến hành lắp đặt hệ thống tại phòng để chạy thực nghiệm, sau đó sử dụng các phương pháp đánh giá để kiểm tra độ hiệu quả và chính xác của hệ thống, từ đó biết được các ưu, nhược điểm của sản phẩm.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận:

Hệ thống IoT của chúng tôi mặc dù đã hoàn thiện, đã đáp ứng được với đề tài của Đồ án chuyên ngành, tuy nhiên vẫn chưa hoàn thiện được phần thiết bị cơ khí cho end node và tích hợp lên phao cứu sinh tự bơm khí nén. Nên chỉ thực hiện thô sơ end node và đã xác định được vị trí về kinh độ vĩ độ, độ cao sô với mực nước biển và và khoảng cách từ khách du lịch mang thiết bị này đến trạm cứu hộ.Ssau một khoảng thời gian đủ lâu để giám sát và đánh giá thì hệ thống đã đáp ứng hầu hết các yêu cầu được đặt ra từ đầu, cùng với đó là sự ổn định và ảnh hưởng tích cực tới sự an toàn thông tin, an toàn thiết bị trong phòng server của trung tâm. Mặc dù vẫn còn tồn tại nhiều hạn chế kể trên, tuy nhiên hệ thống vẫn được đánh giá tốt và hoạt động đúng với mong đợi của nhóm cũng như người hướng dẫn.

Hướng phát triển của đề tài:

Đây là danh sách ngắn gọn các nhiệm vụ chúng tôi dự định thực hiện trong tương lai để nâng cao sản phẩm:

- Cải thiện nhiều hơn về khoảng cách truyền của end node tới gateway bằng cách cải tiến anttena1.

-Thiết lập các chức năng tích hợp ánh sáng, bơm khí hơi phao, kiểm tra điện áp hoạt động của mạch

- Phát triển sản phẩm cơ khí áo phao khí nén tích hợp end node để đáp ứng đề tài nghiên cứu khoa học sau này

- Xây dựng hệ thống giám sát bằng thiết bị smartphone để linh hoạt trong việc theo dõi

- Xây dựng tích hợp nhiều end node hơn nữa để tạo mạng lưới giám sát, theo dõi cứu hộ cứu nạn đáp ưng cho cách ngành du lịch yêu cầu sự giám sát đảm bảo an toàn.

THAM KHẢO

[1] Bài báo: MẠNG CÔNG NGHIỆP LORAWAN CHO THÀNH PHỐ ĐÀ NẴNG: GIẢI PHÁP CHO ỨNG DỤNG IoT TẦM XA TIÊU THỤ NĂNG LƯỢNG THẤP. Tác giả:Ngô Đình Thanh , Fabien Ferrero , Lê Công Vĩnh Khải , Nguyễn Huỳnh Nhật Thương , Phạm Văn Tuấn , Lê Quốc Huy. Tạp chí khoa học và công nghệ, Đại học Đà Nẵng. ISSN 1859-1531 VOL. 18, NO. 1, 2020

[2] Bài báo: MẠNG KHÔNG DÂY LORA CHO ỨNG DỤNG IOT TẦM XA. Tác giả: *Trần Văn Líc*, *Lê Hồng Nam*. Tạp chí khoa học và công nghệ, Đại học Đà Nẵng. Số: B2018-214. Trang: 50-53. Năm 2018. (Apr 8 2019 2:38PM)

[3] Bài báo: Phân tích ảnh hưởng các thông số mạng LoRaWAN đến hiệu suất hoạt động của mạng trong môi trường thực tế. Tác giả: Trần Văn Líc, Tô Đông Vĩ. Tạp chí Khoa học và Công nghệ - Đại học Đà Nẵng. Số: ISSN: 1859-1531. Số: 20. Trang: 00. Năm 2021. (May 23 2021 9:37AM)

[4] https://tapit.vn/tiep-can-voi-cong-nghe-truyen-thong-khong-day-lora/
https://lora-developers.semtech.com/library/technical-documents/

[5] Github.com.

[6] Tài liệu tham khảo của Thầy Vũ Vân Thanh.

PHỤ LỤC 1

```
#include <lmic.h>
#include <SoftwareSerial.h>
#include <hal/hal.h>
#include <SPI.h>
#include <TinyGPS.h>
#include <EEPROM.h>
static const int RXPin = 4, TXPin = 5;
static const uint32_t GPSBaud = 9600;
TinyGPS gps;
// The serial connection to the GPS device
SoftwareSerial ss(RXPin, TXPin);
const unsigned long interval = 1000;    // 10 s interval to send message
unsigned long previousMillis = 0;  // will store last time message sent
const unsigned TX_INTERVAL = 10;
const int nhapnhay = A3;//chân reset
const int ledpin =6;//chân reset
int L_RSSI;
static const PROGMEM u1_t NWKSKEY[16] = {0xAB, 0xD2, 0x62, 0xEF, 0xAF,
   0xA4, 0xAA, 0x57, 0xF5, 0x12, 0x9B, 0xD9, 0xC0, 0x34, 0xA6, 0x87};
static const u1_t PROGMEM APPSKEY[16] = {0xA2, 0x69, 0x85, 0x6B, 0x4C,
   0x27, 0x8E, 0xA7, 0x68, 0xAB, 0x36, 0xA6, 0x8A, 0x04, 0x15, 0x73};
static const u4_t DEVADDR = 0x260BE6FD;
void os_getArtEui (u1_t* buf) { }
void os_getDevEui (u1_t* buf) { }
void os_getDevKey (u1_t* buf) { }

static void smartdelay(unsigned long ms);

unsigned int count = 1;        //For times count


String datastring1="";
String datastring2="";
String datastring3="";
String datastring4="";
uint8_t mydata[9];     //Used to store GPS data for uploading

char gps_lon[20]={"\0"};  //Storage GPS info
```

```
char gps_lat[20]={"\0"}; //Storage latitude
char gps_alt[20]={"\0"}; //Storage altitude
char gps_dis[20]={"\0"}; //Storage altitude
float flat, flon,falt, fdis;

static osjob_t sendjob;
const lmic_pinmap lmic_pins = {
    .nss = 10,
    .rxtx = LMIC_UNUSED_PIN,
    .rst = 8,
    .dio = {2, 7, 9},
};
void onEvent (ev_t ev) {
    Serial.print(os_getTime());
    Serial.print(": ");
    switch(ev) {
        case EV_SCAN_TIMEOUT:
            Serial.println(F("EV_SCAN_TIMEOUT"));
            break;
        case EV_BEACON_FOUND:
            Serial.println(F("EV_BEACON_FOUND"));
            break;
        case EV_BEACON_MISSED:
            Serial.println(F("EV_BEACON_MISSED"));
            break;
        case EV_BEACON_TRACKED:
            Serial.println(F("EV_BEACON_TRACKED"));
            break;
        case EV_JOINING:
            Serial.println(F("EV_JOINING"));
            break;
        case EV_JOINED:
            Serial.println(F("EV_JOINED"));
            break;
        case EV_RFU1:
            Serial.println(F("EV_RFU1"));
            break;
        case EV_JOIN_FAILED:
            Serial.println(F("EV_JOIN_FAILED"));
```

```
            break;
        case EV_REJOIN_FAILED:
            Serial.println(F("EV_REJOIN_FAILED"));
            break;
        case EV_TXCOMPLETE:
            Serial.println(F("EV_TXCOMPLETE (includes waiting for RX windows)"));
            //if (LMIC.txrxFlags & TXRX_ACK)
              //*Serial.println(F("Received ack"));
            if (LMIC.dataLen) {
              Serial.println(F("Received "));
              Serial.println(LMIC.dataLen);
              Serial.println(F(" bytes of payload"));
            }
            // Schedule next transmission
            os_setTimedCallback(&sendjob, os_getTime()+sec2osticks(TX_INTERVAL),
    do_send);
            break;
        case EV_LOST_TSYNC:
            Serial.println(F("EV_LOST_TSYNC"));
            break;
        case EV_RESET:
            Serial.println(F("EV_RESET"));
            break;
        case EV_RXCOMPLETE:
            // data received in ping slot
            Serial.println(F("EV_RXCOMPLETE"));
            break;
        case EV_LINK_DEAD:
            Serial.println(F("EV_LINK_DEAD"));
            break;
        case EV_LINK_ALIVE:
            Serial.println(F("EV_LINK_ALIVE"));
            break;
         default:
            Serial.println(F("Unknown event"));
            break;
    }
}
void do_send(osjob_t* j)
```

```
  {

    if (LMIC.opmode & OP_TXRXPEND)
    {
       //Serial.println(F("OP_TXRXPEND, error send data"));
    } else
    {
       LMIC_setTxData2(1, mydata, sizeof(mydata)-1, 1);
       GPSRead();
       GPSWrite();
    }
    Serial.print("====>>>>>> Truyen data tai Kenh: ");Serial.println(LMIC.txChnl);
     LMIC.rssi=0;
  }

  void GPSRead()
{
  unsigned long age;
   static const double LONDON_LAT =  16.072808536766136, LONDON_LON =
    108.22629464094483;
  gps.f_get_position(&flat, &flon, &age);
  falt=gps.f_altitude();  //get altitude
  flon == TinyGPS::GPS_INVALID_F_ANGLE ? 0.0 : flon, 6;//save six decimal
    places
  flat == TinyGPS::GPS_INVALID_F_ANGLE ? 0.0 : flat, 6;
// falt == TinyGPS::GPS_INVALID_F_ANGLE ? 0.0 : falt, 2;
    fdis = ((unsigned long)TinyGPS::distance_between(flat, flon, LONDON_LAT,
    LONDON_LON), 0xFFFFFFFF, 9);
  //Serial.println(flat);
   Serial.print("");
  Serial.print(fdis);
}

void GPSWrite()
{
  /*Convert GPS data to format*/
  datastring1 +=dtostrf(flat, 0, 6, gps_lat);
  datastring2 +=dtostrf(flon, 0, 6, gps_lon);
  datastring3 +=dtostrf(falt, 0, 2, gps_alt);
```

```
datastring4 +=dtostrf(fdis, 0, 9, gps_dis);


if(flon!=1000.000000)
{
strcat(gps_lat,",");
strcat(gps_lat,gps_lon);
//strcat(gps_lon,",");
//strcat(gps_lon,gps_alt);
strcpy((char*)mydata,gps_lat); //the format of mydata is longtitude,latitude,altitude
Serial.print("###########    ");
Serial.print("NO.");
Serial.print(count);
Serial.println("    ###########");
Serial.println("The longtitude and latitude are:");
Serial.print("[");
Serial.print((char*)mydata);
Serial.print("]");
Serial.print("");

count++;
}

  int32_t lat = flat*10000;
  int32_t lng= flon*10000;
   int32_t alt= falt;
  int32_t dis=fdis*1000;

mydata[0] = (lng&0xFF);
mydata[1] = (lng >> 8);
mydata[2] = (lng>>16) ;

mydata[3] = (lat&0xFF);
mydata[4] = (lat >> 8);
mydata[5] = (lat>>16 );
mydata[6]= alt&0xFF;
mydata[7]=  alt >>8;
mydata[6]= dis&0xFF;
mydata[7]=  dis >>8;
```

```
  smartdelay(1000);
  //Serial.println(gps_lat);
//Serial.println(gps_lon);
}


static void smartdelay(unsigned long ms)
{
  unsigned long previousMillis = millis();
  do
  {
    while (ss.available())
    {
      gps.encode(ss.read());
    }
  } while (millis() - previousMillis < ms);
}



void setup()
{
    Serial.begin(115200);
  ss.begin(GPSBaud);
   digitalWrite(ledpin,HIGH);
   pinMode(ledpin, OUTPUT);
   pinMode(nhapnhay, OUTPUT);
   digitalWrite(nhapnhay,LOW);
    #ifdef VCC_ENABLE
    pinMode(VCC_ENABLE, OUTPUT);
    digitalWrite(VCC_ENABLE, HIGH);
    delay(1000);
   delay(1000);
    #endif
   // wdt_enable(WDTO_4S);
    os_init();
    LMIC_reset();
    LMIC_setClockError(MAX_CLOCK_ERROR * 10 / 100);
```

```
#ifdef PROGMEM
uint8_t appskey[sizeof(APPSKEY)];
uint8_t nwkskey[sizeof(NWKSKEY)];
memcpy_P(appskey, APPSKEY, sizeof(APPSKEY));
memcpy_P(nwkskey, NWKSKEY, sizeof(NWKSKEY));
LMIC_setSession (0x1, DEVADDR, nwkskey, appskey);
#else
LMIC_setSession (0x1, DEVADDR, NWKSKEY, APPSKEY);
#endif
LMIC_setupChannel(0, 923200000, DR_RANGE_MAP(DR_SF12, DR_SF7),
BAND_CENTI);      // g-band
// LMIC_setupChannel(1, 923400000, DR_RANGE_MAP(DR_SF12, DR_SF7B),
BAND_CENTI);      // g-band
 // LMIC_setupChannel(2, 922200000, DR_RANGE_MAP(DR_SF12, DR_SF7),
BAND_CENTI);      // g-band
 // LMIC_setupChannel(3, 922400000, DR_RANGE_MAP(DR_SF12, DR_SF7),
BAND_CENTI);      // g-band
 // LMIC_setupChannel(4, 922600000, DR_RANGE_MAP(DR_SF12, DR_SF7),
BAND_CENTI);      // g-band
 // LMIC_setupChannel(5, 922800000, DR_RANGE_MAP(DR_SF12, DR_SF7),
BAND_CENTI);      // g-band
 // LMIC_setupChannel(6, 923000000, DR_RANGE_MAP(DR_SF12, DR_SF7),
BAND_CENTI);      // g-band
 // LMIC_setupChannel(7, 922000000, DR_RANGE_MAP(DR_SF12, DR_SF7),
BAND_CENTI);      // g-band
 // LMIC_setupChannel(8, 922100000, DR_RANGE_MAP(DR_FSK,  DR_FSK),
BAND_MILLI);      // g2-band
LMIC_disableChannel(3);
LMIC_disableChannel(1);
LMIC_disableChannel(2);
LMIC_disableChannel(6);
LMIC_disableChannel(4);
LMIC_disableChannel(5);
LMIC_disableChannel(7);
LMIC_disableChannel(8);
LMIC_setLinkCheckMode(0);
LMIC.dn2Dr = DR_SF10;
LMIC_setDrTxpow(DR_SF7,14);
//LMIC_enableTracking(0);
```

```
  do_send(&sendjob);
  GPSRead();
      GPSWrite();

}
void(* resetFunc) (void) = 0;//cài đặt hàm reset
void loop() {
      if(millis() - previousMillis > interval)
  {  previousMillis = millis();


  os_setTimedCallback(&sendjob, 1, do_send);
  }


os_runloop_once();
}
```